NIET Greater Noida
Autonomous Institute
Estd.2001
Get Future Ready

Syllabus

Course Contents / Syllabus		
Module 1	**INTRODUCTION TO MACHINE LEARNING**	**10 hours**
Learning, Types of Learning, well defined learning problems, designing a Learning System, History of ML, Introduction of Machine Learning Approaches, Features and their types, handing missing data, Dealing with categorical features, Feature Scaling, Feature selection, outlier detection and removal. Introduction to Model Building, Sensitivity Analysis, Underfitting and Overfitting, Bias and Variance, Activation Functions, Data Science Vs Machine Learning.		
Module 2	**CLASSIFICATION**	**18 hours**
Classification: Classification problem, Binary and Multiclass Classification, Decision Boundary **Measuring Performance of Classification Model:** Accuracy, Precision, Recall, F1-Score, ROC-AUC curve, Confusion matrix. **Decision Trees:** Decision Tree Classification Algorithm, Attribute Selection Measures, Variants of Decision Trees. **Large Margin Classifier:** Support Vector Machines, **Instance Based Learning:** K Nearest Neighbor Algorithm.		
Module 3	**REGRESSION**	**12 hours**
Logistic regression model for binary classification, Cost function of logistic regression model, Regularized logistic regression, One Vs. All strategy for multiclass classification using logistic regression. **Introduction to Regression:** Regression Problem, Linear regression model (Simple, Multiple and Polynomial), **Errors Measures:** Mean Squared Error (MSE), Root Mean Squared Error (RMSE), Mean Absolute Error (MAE), **Cost function:** Formula, intuition and visualization, Bias and Variance, Overfitting		

Syllabus

and Underfitting, **Gradient Descent:** Algorithm, intuition, learning rate and Gradient Descent for linear regression.		
Module 4	**PROBABILISTIC & CLUSTERING TECHNIQUES**	**12 hours**
Probabilistic Methods for Classification: Bayesian Learning, Naive Bayes Classifier, Bayesian belief networks. **Tree Ensembles:** Using multiple decision trees, Bagging Vs. Boosting, Sampling with replacement, Random Forest Algorithm, Adda boost, Gradient Boosting, Xgboost Algorithm. **Unsupervised Learning &Clustering:** The problem of Feed, Distance measures **Partition Based Methods:** K-Means Clustering (Intuition, Algorithm, Optimization objective, Initializing K-Means, Choosing number of clusters) K-Medoids and K-Mode clustering algorithms. **Hierarchical Clustering** (Single Linkage, Complete Linkage, AGNES (Agglomerative Nesting), DIANA (Divisive Analysis). Reinforcement Learning		
Module 5	**ARTIFICIAL NEURAL NETWORKS**	**8 hours**
Neural Networks: Introduction, Perceptron, Multilayer Perceptron, Feed Forward Networks, Back Propagation Networks, Back-propagation through time (BTT), Different types of RNNs. Recurrent Neural Network Model, Notation, hyper-parameter tuning, Image classification, Text classification, Image classification. Understanding and visualizing a CNN. Case Study: Health Care, E Commerce, Smart Cities.		
Total Lecture Hours		**60 hours**

Branch Wise Applications

Sample Applications

- Web search
- Computational biology
- Finance
- E-commerce
- Space exploration
- Robotics
- Information extraction
- Social networks
- Debugging software
- [Your favorite area]

Prerequisites:

- Statistics.
- Linear Algebra.
- Calculus.
- Probability.
- Programming Languages.

NIET
Greater Noida
Autonomous Institute
Estd.2001
Get Future Ready

Content - Unit 2

Classification: Classification problem, Binary and Multiclass Classification, Decision Boundary
Measuring Performance of Classification Model: Accuracy, Precision, Recall, F1-Score, ROC-AUC curve,
Confusion matrix. **Decision Trees:** Decision Tree Classification Algorithm, Attribute Selection Measures, Variants of
Decision Trees. **Large Margin Classifier:** Support Vector Machines, **Instance Based Learning:** K Nearest
Neighbor Algorithm.

Unit Objective

- **Classification Fundamentals**
  - Classification problem & supervised learning
  - Binary vs Multiclass vs Multi-label
  - Decision Boundary visualization
- **Performance Metrics**
  - Confusion Matrix (TP, TN, FP, FN)
  - Accuracy, Precision, Recall, F1-Score
  - ROC curve & AUC
- **Decision Trees**
  - DT algorithm & terminology
  - Attribute selection: Entropy, Information Gain, Gini
  - Variants: ID3, C4.5, CART
- **Large Margin Classifier**
  - Support Vector Machines (SVM)
  - Hard / Soft margin, Kernel trick
- **Instance-Based Learning**
  - K-Nearest Neighbor (KNN)
  - Distance metrics, choosing K

Topic Objective

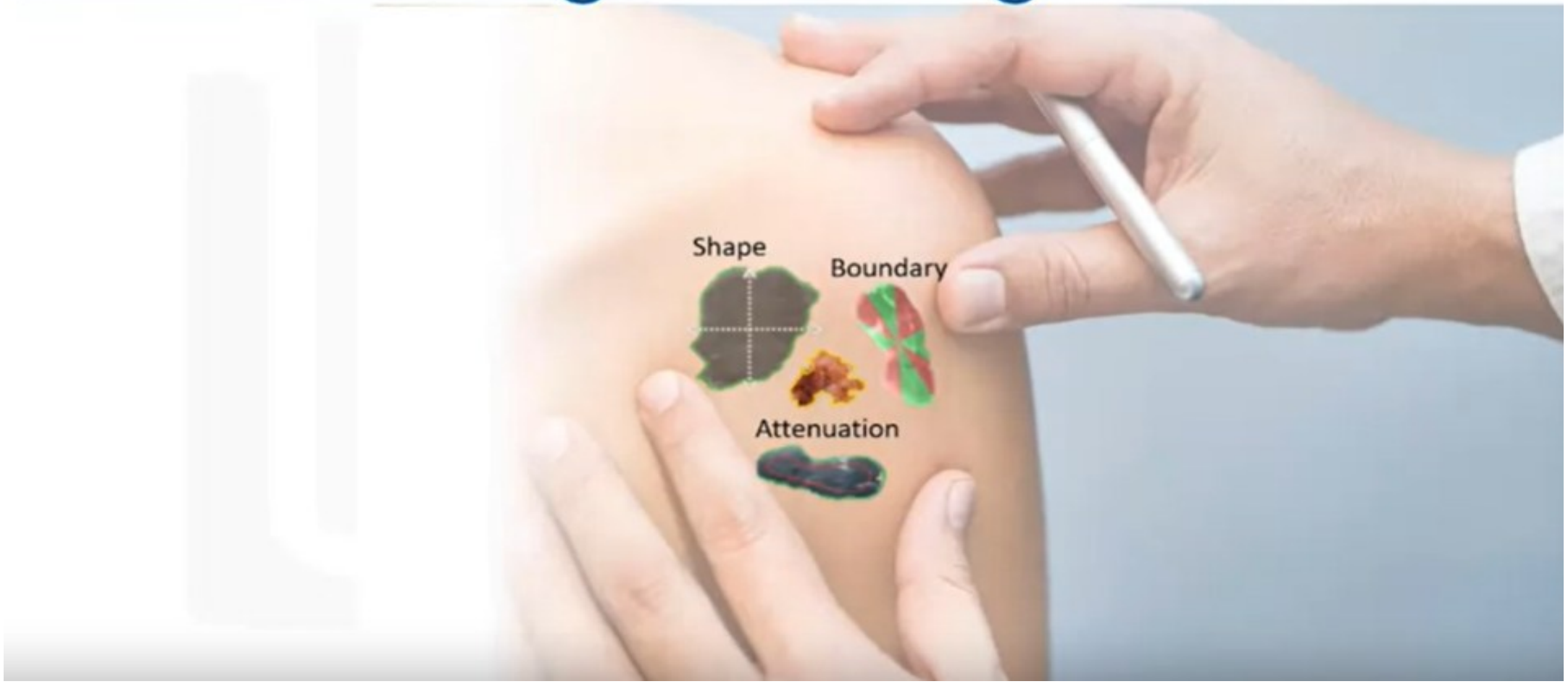
Student will be able to understand

- Classification
- Performance evaluation
- Types of classification

NIET
Greater Noida
Autonomous Institute
Estd.2001
Get Future Ready

Classification

Is this a benign or malignant cell?



Classification

Machine learning helps with predictions!

ID	Clump	UnifSize	UnifShape	MargAdh	SingEpiSize	BareNuc	BlandChrom	NormNucl	Mit	Class
1000025	5	1	1	1	2	1	3	1	1	benign
1002945	5	4	4	5	7	10	3	2	1	benign
1015425	3	1	1	1	2	2	3	1	1	malignant
1016277	6	8	8	1	3	4	3	7	1	benign
1017023	4	1	1	3	2	1	3	1	1	benign
1017122	8	10	10	8	7	10		7	1	malignant
1018099	1	1	1	1	2	10	3	1	1	benign
1018561	2	1	2	H	2	1	3	1	1	benign
1033078	2	1	1	1	2	1	1	1	5	benign
1033078	4	2	1	1	2	1	2	1	1	benign

Classification

Machine learning helps with predictions!

ID	Clump	UnifSize	UnifShape	MargAdh	SingEpiSize	BareNuc	BlandChrom	NormNucl	Mit	Class
1000025	5	1	1	1	2	1	3	1	1	benign
1002945	5	4	4	5	7	10	3	2	1	benign
1015425	3	1	1	1	2	2	3	1	1	malignant
1016277	6	8	8	1	3	4	3	7	1	benign
1017023	4	1	1	3	2	1	3	1	1	benign
1017122	8	10	10	8	7	10		7	1	malignant
1018099	1	1	1	1	2	10	3	1	1	benign
1018561	2	1	2	H	2	1	3	1	1	benign
1033078	2	1	1	1	2	1	1	1	5	benign
1033078	4	2	1	1	2	1	2	1	1	benign

Modeling

ID	Clump	UnifSize	UnifShape	MargAdh	SingEpiSize	BareNuc	BlandChrom	NormNucl	Mit	Class
1000015	6	1	1	1	7	1	3	1	1	Benign

Prediction



Accuracy = 89%

Classification

How does classification work?

Classification determines the class label for an unlabeled test case.

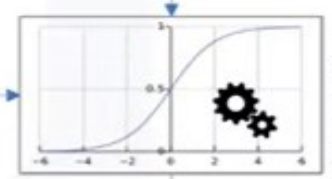
age	ed	employ	address	income	debtinc	creddebt	othdebt	default
41	3	17	12	176	9.3	11.359	5.009	1
27	1	10	6	31	17.3	1.362	4.001	0
40	1	15	14	55	5.5	0.856	2.169	0
41	1	15	14	120	2.9	2.659	0.821	0
24	2	2	0	28	17.3	1.787	3.057	1
41	2	5	5	25	10.2	0.393	2.157	0
39	1	20	9	67	30.6	3.834	16.668	0
43	1	12	11	38	3.6	0.129	1.239	0
24	1	3	4	19	24.4	1.358	3.278	1
36	1	0	13	25	19.7	2.778	2.147	0

Categorical Variable

Modeling

age	ed	employ	address	income	debtinc	creddebt	othdebt	default
37	2	16	10	130	9.3	10.23	3.21	0

Prediction



Classifier

Predicted Labels

NIET Greater Noida
Autonomous Institute
Estd.2001
Get Future Ready

Classification

Definition

Classification is a SUPERVISED learning task where the model learns from LABELLED training data to predict a CATEGORICAL output (class) for new, unseen inputs. Unlike regression (which predicts continuous numbers), classification predicts discrete labels — "spam or not spam", "cat / dog / bird", "benign / malignant".

Real-World Examples

Email Spam Filter

Input: email text → Output: {Spam, Not Spam}

Medical Diagnosis

Input: scan / test → Output: {Healthy, Diseased}

Image Recognition

Input: photo → Output: {Cat, Dog, Bird, ...}

Credit Risk

Input: applicant data → Output: {Approve, Reject}

Sentiment Analysis

Input: review text → Output: {Positive, Neutral, Negative}

Customer Churn

Input: usage history → Output: {Stay, Leave}

Formal goal: Learn a function $f : X \rightarrow Y$ where X is the input space and Y is a finite set of classes $\{y_1, y_2, ..., y_k\}$.
The model assigns each new input to one of the K predefined classes based on patterns learned from training data.

NIET Greater Noida
Autonomous Institute
Estd.2001
Get Future Ready

Classification Workflow / Pipeline

The end-to-end process of building a classification system:

1 — Data Collection

Gather labelled examples (X, y).

2 — Preprocessing

Clean, encode, scale, split into train/test.

3 — Choose Algorithm

Decision Tree, SVM, KNN, NN, ensemble...

4 — Train Model

Fit on training data, tune hyperparams.

5 — Evaluate

Compute accuracy, precision, recall, F1.

6 — Deploy & Monitor

Serve predictions, watch for drift.

Mathematical Framing

Training dataset: $D = \{ (x_1, y_1), (x_2, y_2), \ldots, (x_n, y_n) \}$
Goal: Find $\hat{f}$ that minimizes loss between predicted $\hat{y}$ and true y on UNSEEN data — i.e., GENERALIZES well.
Loss functions: Cross-entropy (most common), Hinge loss (SVM), Log loss, 0-1 loss.
Key principle: *training error ≠ test error. Always evaluate on a held-out test set.*

NIET Greater Noida
Autonomous Institute
Estd.2001
Get Future Ready

Binary Classification

Definition

Binary classification is the simplest case — there are EXACTLY TWO possible classes. Conventionally one is called "positive" (1) and the other "negative" (0). Most theoretical results in classification start with binary because the math is cleaner; many algorithms extend to multiclass via one-vs-rest.

Common Examples

Spam Detection

Positive (1): Spam
Negative (0): Not Spam

Cancer Diagnosis

Positive (1): Malignant
Negative (0): Benign

Loan Approval

Positive (1): Approved
Negative (0): Rejected

Fraud Detection

Positive (1): Fraud
Negative (0): Legitimate

Algorithms Designed for Binary Classification

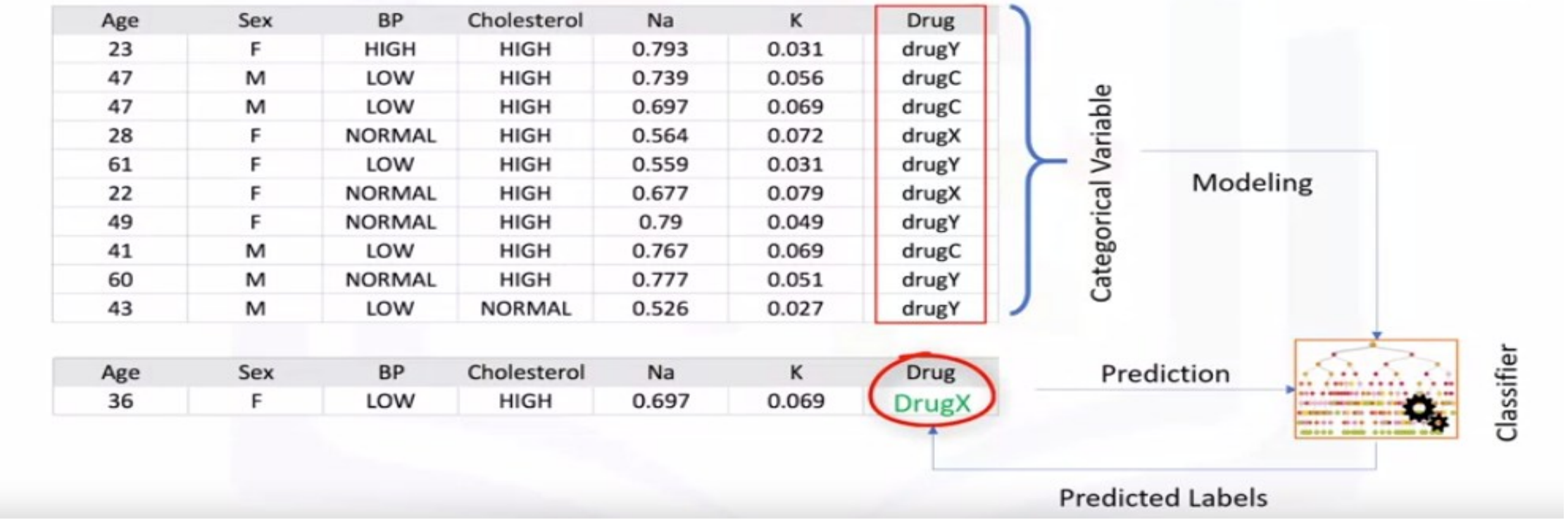
- **Logistic Regression —** outputs probability P(y=1|x) via sigmoid; decision threshold at 0.5.
- **Support Vector Machine (SVM) —** finds maximum-margin hyperplane separating two classes.
- **Perceptron —** the simplest neural network, learns a linear decision boundary.
- **Decision Trees, Random Forest, KNN, Naive Bayes —** all natively handle binary classification.

Multiclass Classification

Example of multi-class classification

Age	Sex	BP	Cholesterol	Na	K	Drug
23	F	HIGH	HIGH	0.793	0.031	drugY
47	M	LOW	HIGH	0.739	0.056	drugC
47	M	LOW	HIGH	0.697	0.069	drugC
28	F	NORMAL	HIGH	0.564	0.072	drugX
61	F	LOW	HIGH	0.559	0.031	drugY
22	F	NORMAL	HIGH	0.677	0.079	drugX
49	F	NORMAL	HIGH	0.79	0.049	drugY
41	M	LOW	HIGH	0.767	0.069	drugC
60	M	NORMAL	HIGH	0.777	0.051	drugY
43	M	LOW	NORMAL	0.526	0.027	drugY

Age	Sex	BP	Cholesterol	Na	K	Drug
36	F	LOW	HIGH	0.697	0.069	DrugX



NIET Greater Noida
Autonomous Institute
Estd.2001
Get Future Ready

Multiclass Classification

Definition

Multiclass classification has MORE THAN TWO mutually-exclusive classes. Each input belongs to EXACTLY ONE class. Examples: handwritten digit recognition (10 classes), iris flower species (3), language detection (50+).

Two Strategies to Extend Binary Classifiers

One-vs-Rest (OvR)

Train K binary classifiers — each one separates ONE class from ALL others.

For 3-class problem (A, B, C):
- Classifier 1: A vs (B, C)
- Classifier 2: B vs (A, C)
- Classifier 3: C vs (A, B)

Predict: *the class whose classifier has the highest confidence.*

One-vs-One (OvO)

Train K(K-1)/2 binary classifiers — ONE for each PAIR of classes.

For 3-class problem (A, B, C):
- Classifier 1: A vs B
- Classifier 2: A vs C
- Classifier 3: B vs C

Predict: *MAJORITY VOTE — the class that wins the most pairwise contests.*

OvR: Fewer classifiers (K), faster training, default in sklearn. **OvO:** Each classifier sees less data, preferred for SVMs with many classes.

NIET Greater Noida
Autonomous Institute
Estd.2001
Get Future Ready

Multi-Label vs Multi-Class

Easy to Confuse — Critical to Distinguish!

Multi-CLASS: input belongs to EXACTLY ONE class out of many. Multi-LABEL: input can simultaneously belong to MULTIPLE classes. The choice affects loss function, output layer, and evaluation metrics.

Multi-CLASS

Exactly ONE label per input.

Classes are MUTUALLY EXCLUSIVE.

Example: Iris flower species

```
Input: flower features
Output: Setosa OR Versicolor OR Virginica
```

Output layer:
Softmax (probabilities sum to 1).

Loss function:
Categorical cross-entropy.

Encoding:
One-hot vector (one 1, rest 0).

Multi-LABEL

Zero, one, or MANY labels per input.

Classes are INDEPENDENT.

Example: Movie genres

```
Input: movie description
Output: {Action, Comedy, Romance}
```

Output layer:
Sigmoid PER class (independent probs).

Loss function:
Binary cross-entropy (per label).

Encoding:
Multi-hot vector (many 1s allowed).

NIET Greater Noida
Autonomous Institute
Estd.2001
Get Future Ready

Decision Boundary

Definition

A decision boundary is the SURFACE in the feature space that SEPARATES different classes. Points on one side are classified one way; points on the other side, the other way. The COMPLEXITY of this boundary is what differentiates simple linear classifiers from sophisticated non-linear ones.

Linear vs Non-Linear Boundaries

Linear Decision Boundary

A straight line (2D) or hyperplane (higher dim).

Equation:

$$w_1x_1 + w_2x_2 + \ldots + b = 0$$

Algorithms:

- Logistic Regression
- Linear SVM
- Perceptron
- LDA

Limitation: *fails on non-linearly separable data (e.g. XOR).*

Non-Linear Decision Boundary

A curve, circle, or complex surface.

Examples:
Quadratic, polynomial, piecewise.

Algorithms:

- Decision Trees (piecewise)
- Kernel SVM (RBF)
- KNN
- Neural Networks

Trade-off: *more expressive but higher risk of overfitting.*

NIET Greater Noida
Autonomous Institute
Get Future Ready

Performance Metrics — Why They Matter

Why Accuracy Alone Isn't Enough

Imagine a cancer-detection model on a dataset where 99% of patients are healthy. A model that ALWAYS predicts "healthy" achieves 99% accuracy — but catches ZERO actual cancers. We need richer metrics that capture which kinds of mistakes a model makes and in what proportion.

The Six Core Metrics

Confusion Matrix

Base table showing TP, TN, FP, FN.
Foundation for all other metrics.

Accuracy

Fraction of correct predictions.
Simple but misleading for imbalance.

Precision

Of those predicted positive,
how many are actually positive?

Recall (Sensitivity)

Of actual positives,
how many did we catch?

F1-Score

Harmonic mean of
precision and recall.

ROC-AUC

Plot TPR vs FPR across thresholds.
AUC summarizes overall quality.

NIET Greater Noida
Autonomous Institute
Estd.2001
Get Future Ready

Confusion Matrix

Definition

A confusion matrix is a 2×2 table (for binary classification) that compares predicted labels to actual labels — making it visually clear which kinds of mistakes the model makes.

The Binary Confusion Matrix

	Predicted: POSITIVE	Predicted: NEGATIVE
Actual: POSITIVE	**TRUE POSITIVE (TP)** **Correctly predicted positive**	**FALSE NEGATIVE (FN)** **Missed positive (Type II)**
Actual: NEGATIVE	**FALSE POSITIVE (FP)** **False alarm (Type I)**	**TRUE NEGATIVE (TN)** **Correctly predicted negative**

Type I vs Type II Errors

Type I (FP): Predicted positive when actually negative — false alarm. Example: spam filter marks important email as spam.
Type II (FN): Predicted negative when actually positive — missed detection. Example: cancer screen misses a real tumour.

NIET Greater Noida
Autonomous Institute
Estd.2001
Get Future Ready

Accuracy

Definition

Accuracy is the fraction of predictions the model got CORRECT. The most intuitive metric, but can be MISLEADING when classes are imbalanced.

$$\text{Accuracy} = (TP + TN) / (TP + TN + FP + FN)$$

Worked Example — Email Spam Classifier

Suppose model tested on 1000 emails:

TP = 80 (spam correctly flagged)
TN = 870 (ham correctly kept)
FP = 20 (ham wrongly flagged)
FN = 30 (spam missed)

Calculation:

```
Accuracy = (80 + 870) / 1000
         = 950 / 1000
         = 0.95  (95%)
```

When Accuracy Misleads

Cancer dataset: 990 healthy + 10 cancerous patients (severe imbalance). A trivial "predict healthy always" model gets:

Accuracy = 990 / 1000 = 99% — but catches ZERO cancers (0% recall). High accuracy hides catastrophic failure.

Use Precision, Recall, F1, or AUC when classes are imbalanced.

Precision

Definition

Precision answers: "Of all the inputs the model predicted as POSITIVE, how many were actually positive?" It measures the QUALITY of positive predictions — how trustworthy is the model when it says "positive"?

$$\text{Precision} = TP / (TP + FP)$$

When to Optimize for Precision

When **FALSE POSITIVES are costly.** You want HIGH confidence that each positive prediction is truly positive.

Examples:

- Spam filter — wrongly flagging important email is worse than missing some spam.
- Search engines — top results should be truly relevant.
- Drug screening — false alarm wastes time and resources.

Worked Example

From the spam classifier (TP=80, FP=20):

```
Precision = 80 / (80 + 20) = 80 / 100 = 0.80  (80%)
```

Interpretation: *When the model flags an email as spam, it's correct 80% of the time.*

Recall (Sensitivity)

Definition

Recall (also called Sensitivity or True Positive Rate) answers: "Of all the ACTUAL POSITIVES, how many did the model catch?" It measures the COVERAGE of positive cases — how good is the model at finding everything it should find?

$$\text{Recall} = TP / (TP + FN)$$

When to Optimize for Recall

When **FALSE NEGATIVES are costly.** You want to MISS NOTHING, even at the cost of some false alarms.

Examples:

- Cancer screening — missing a tumour is fatal; better to over-test.
- Fraud detection — missing a fraud costs millions.
- Safety alerts — earthquake detection, intrusion alarms.

Worked Example

From the spam classifier (TP=80, FN=30):

```
Recall = 80 / (80 + 30) = 80 / 110 ≈ 0.727  (72.7%)
```

Interpretation: *Of all the actual spam emails, the model successfully catches 72.7% — but misses 27.3%.*

NIET Greater Noida
Autonomous Institute
Estd.2001
Get Future Ready

F1-Score

Definition — Why a Single Metric?

F1-Score is the HARMONIC MEAN of precision and recall. It gives a single number that balances both — useful when you care about both false positives and false negatives. The harmonic mean penalizes extreme values, so a low recall or low precision drags F1 down sharply.

Formula

$$F1 = 2 \cdot (Precision \cdot Recall) / (Precision + Recall)$$

General form (Fβ-score): $F\beta = (1 + \beta^2) \cdot (P \cdot R) / (\beta^2 \cdot P + R)$

F-score Variants

F1 (β=1)

Equal weight to precision and recall.
Default choice.

F0.5 (β=0.5)

Weights PRECISION more.
False positives are worse.

F2 (β=2)

Weights RECALL more.
False negatives are worse.

From spam classifier: P=0.80, R=0.727 → F1 = 2·(0.80·0.727)/(0.80+0.727) ≈ 0.762

NIET Greater Noida
Autonomous Institute
Estd.2001
Get Future Ready

ROC Curve

Definition — Receiver Operating Characteristic

Most classifiers output a PROBABILITY between 0 and 1. The threshold for classifying as "positive" can be moved (default 0.5). The ROC curve plots TPR vs FPR across ALL possible thresholds — showing the trade-off between catching positives and raising false alarms.

The Two Axes

Y-axis: True Positive Rate (TPR)

$$TPR = TP / (TP + FN) = Recall$$

Fraction of actual positives correctly identified.

X-axis: False Positive Rate (FPR)

$$FPR = FP / (FP + TN)$$

Fraction of actual negatives wrongly flagged as positive.

How to Read the ROC Curve

- **Diagonal line (TPR = FPR):** RANDOM classifier — no discrimination ability.
- **Curve above diagonal:** BETTER than random. The further to the top-left corner, the better.
- **Top-left corner (TPR=1, FPR=0):** PERFECT classifier — catches all positives, no false alarms.
- **Below diagonal:** WORSE than random — flip the predictions.

NIET Greater Noida
Autonomous Institute
Estd.2001
Get Future Ready

AUC — Area Under ROC Curve

Definition

AUC is the area under the ROC curve — a SINGLE NUMBER between 0 and 1 that summarizes overall classifier quality across ALL thresholds. AUC has a probabilistic interpretation: it's the probability that the model ranks a random positive higher than a random negative.

AUC Interpretation Scale

AUC Range	Quality	Interpretation
1.0	**Perfect**	Flawless ranking — every positive scored higher than every negative.
0.9 – 1.0	**Excellent**	Very strong discrimination. Production-ready.
0.8 – 0.9	**Good**	Useful in most applications.
0.7 – 0.8	**Fair**	Acceptable for some tasks; needs improvement.
0.5	**Random**	No better than a coin flip — useless.
< 0.5	**Worse than random**	Flip the predictions — the model has learned the OPPOSITE.

Why AUC is popular: threshold-independent, robust to class imbalance, single intuitive number for comparing models.

AUC — Area Under ROC Curve

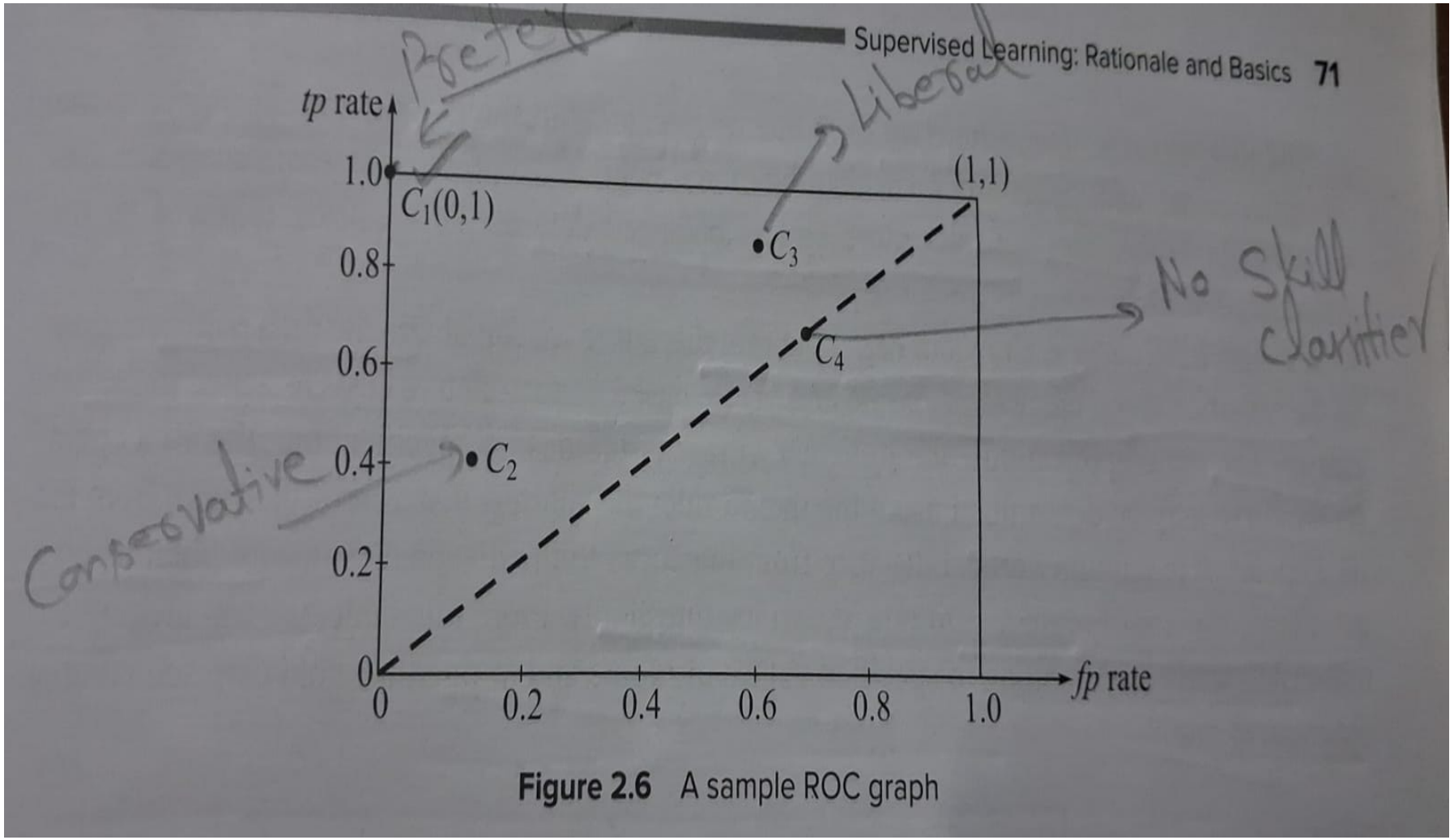


Figure 2.6 A sample ROC graph

NIET
Greater Noida
Autonomous Institute
Estd.2001
Get Future Ready

AUC — Area Under ROC Curve

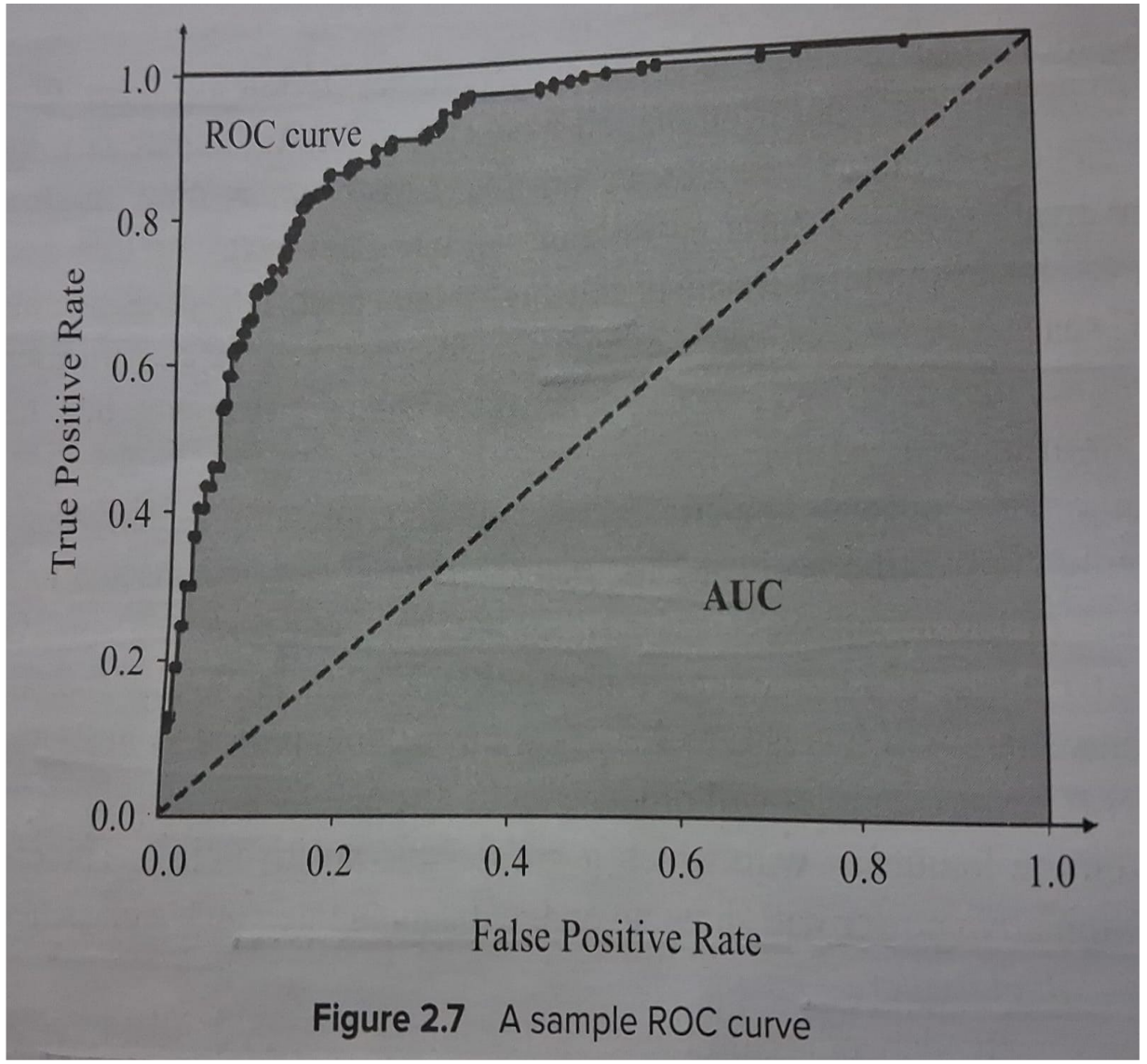
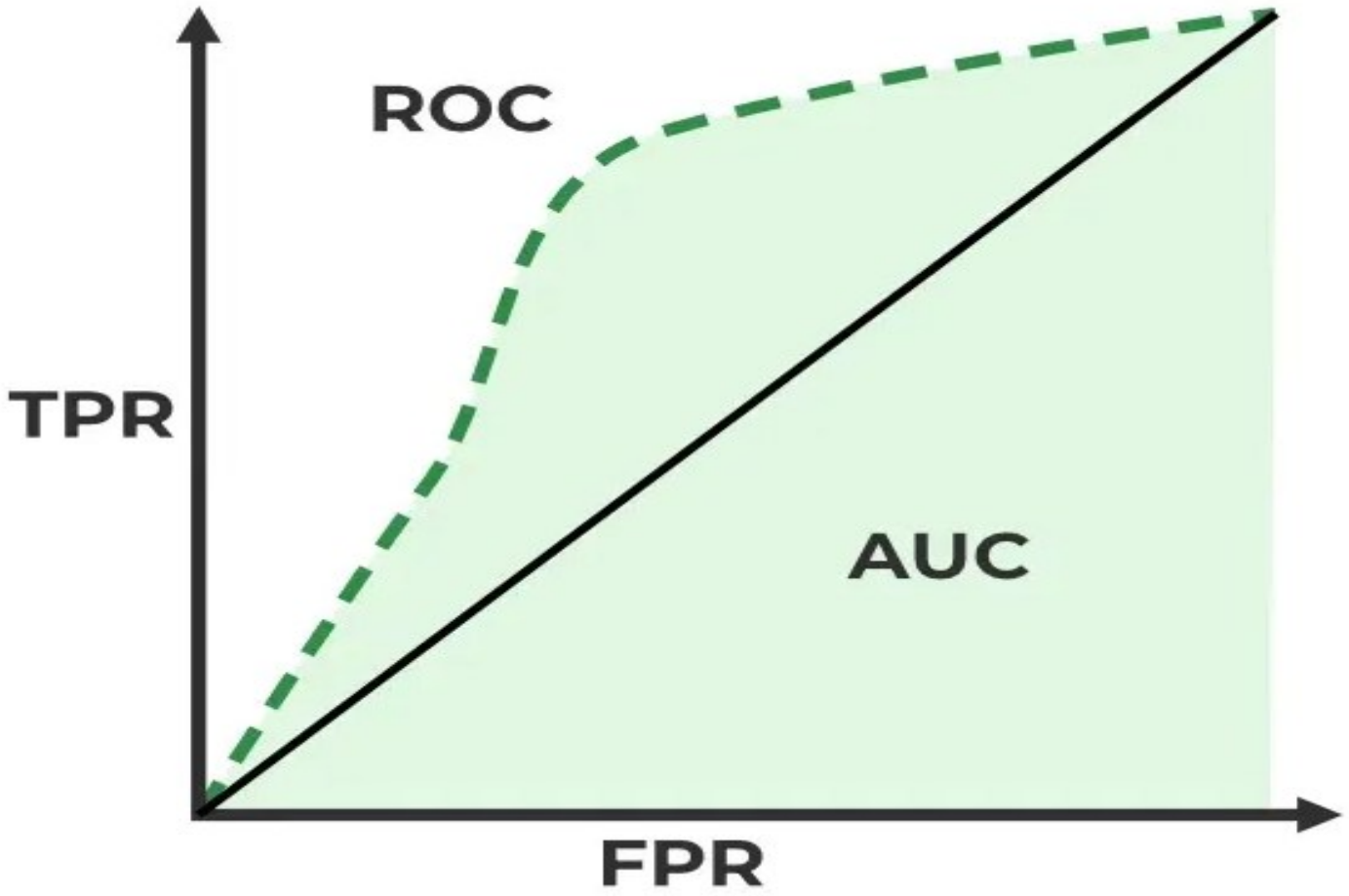
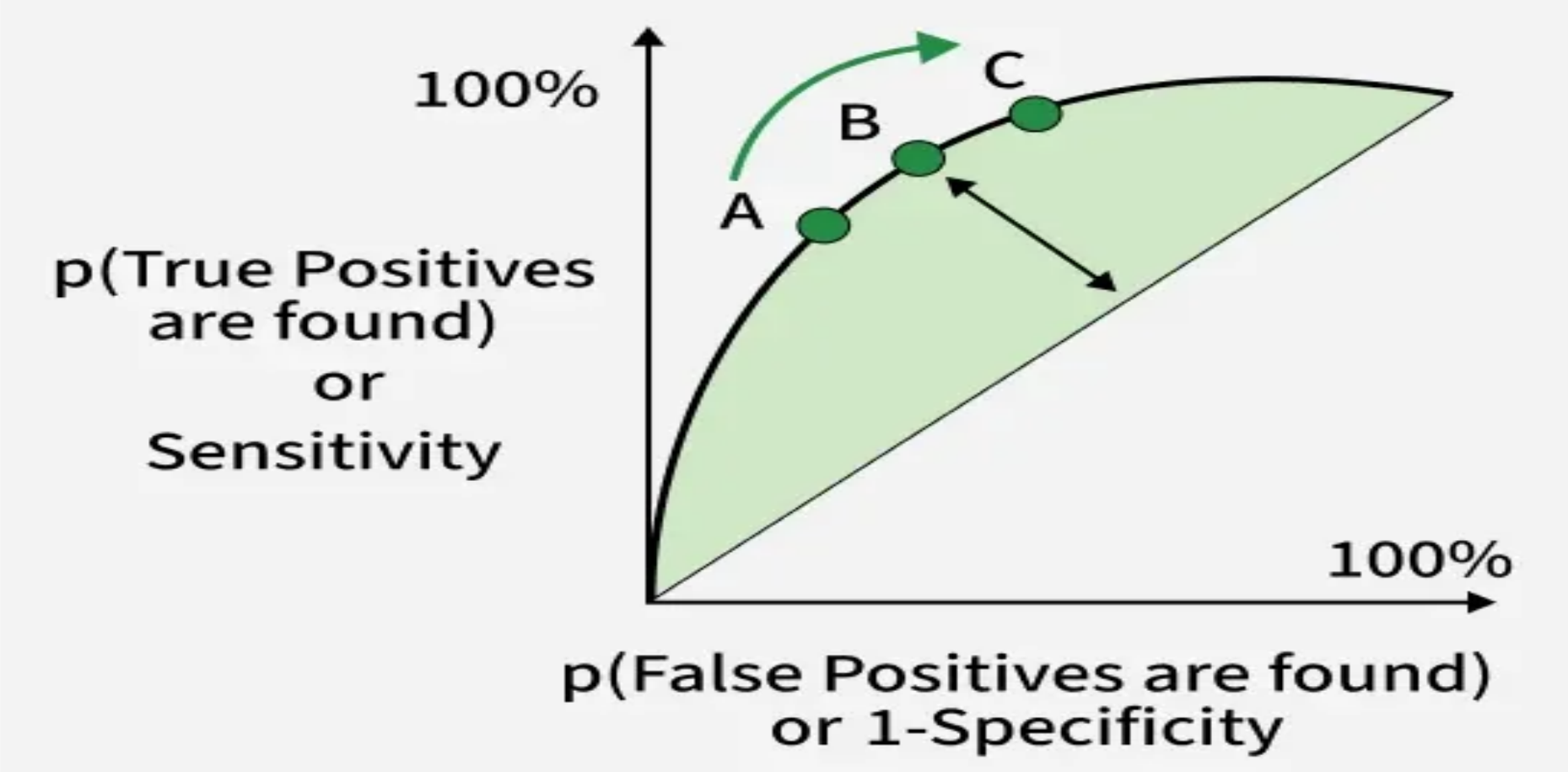


Figure 2.7 A sample ROC curve

AUC — Area Under ROC Curve

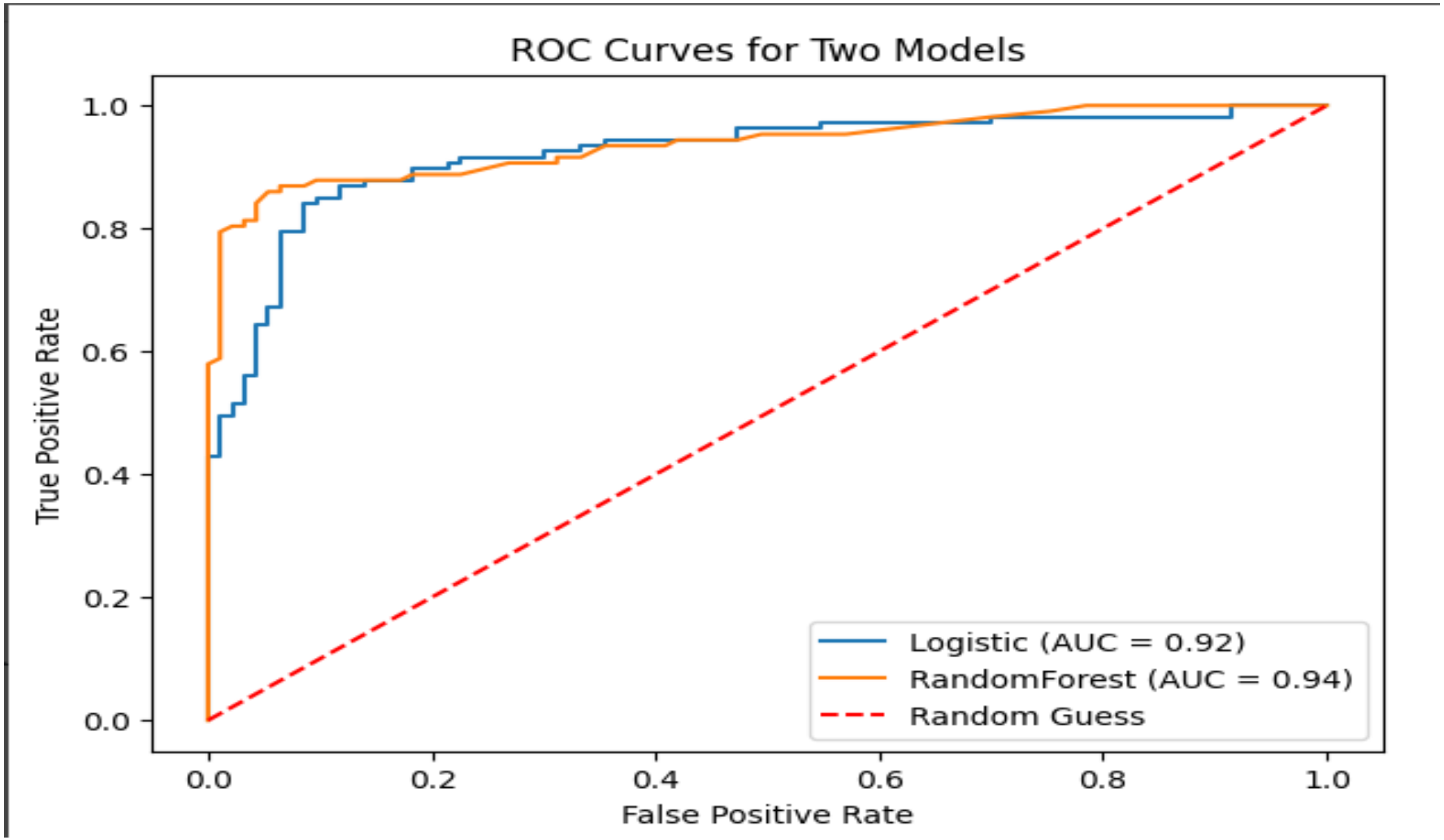


AUC — Area Under ROC Curve



Sensitivity versus False Positive Rate plot

AUC — Area Under ROC Curve



NIET Greater Noida
Autonomous Institute
Estd.2001
Get Future Ready

Decision Trees — Introduction

Definition

A Decision Tree is a supervised learning algorithm that creates a TREE-LIKE structure of decisions to classify (or regress) inputs. At each internal NODE, the data is split on a feature condition; LEAVES contain the final class prediction. Trees mimic human decision-making and are highly interpretable.

Why Decision Trees?

Interpretable

Easy to visualize and explain to stakeholders. "If income > 50k AND credit_score > 700 THEN approve".

No Scaling Needed

Splits on thresholds — feature scales don't matter. Robust to outliers in features.

Handles Both Types

Numerical AND categorical features natively, without encoding.

Non-Linear

Can model complex non-linear relationships via piecewise splits.

Real-World Use Cases

- Credit risk assessment — banks use trees to explain loan decisions to applicants.
- Medical diagnosis — clinical decision support systems.
- Customer churn — telecom and SaaS companies predict who will leave.
- *Foundation for ensemble methods: Random Forest, Gradient Boosting, XGBoost — all built on decision trees.*

Decision Tree

Types of Decision Trees

Types of decision trees are based on the type of target variable we have. It can be of two types:

1. **Categorical Variable Decision Tree:** Decision Tree which has a categorical target variable then it called a **Categorical variable decision tree.**
2. **Continuous Variable Decision Tree:** Decision Tree has a continuous target variable then it is called **Continuous Variable Decision Tree.**

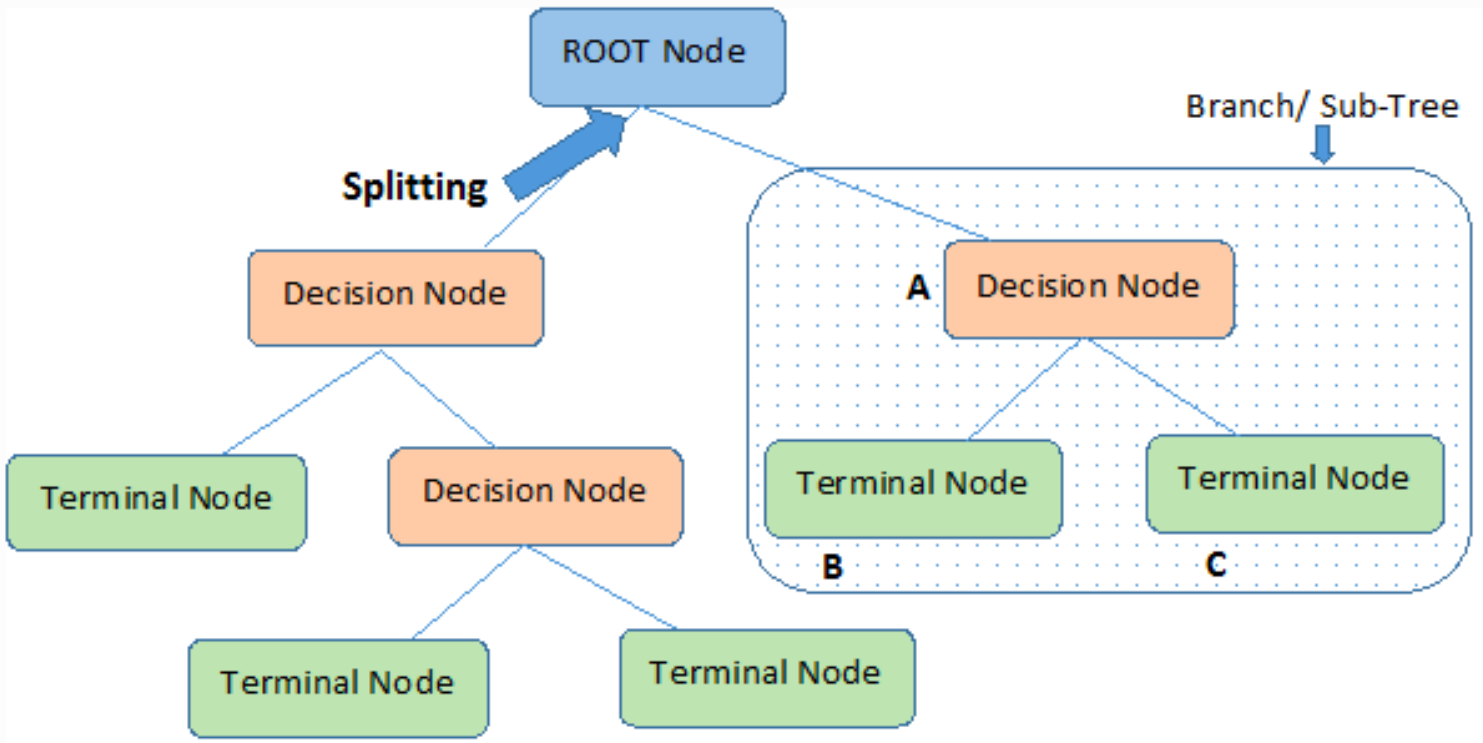
Decision Tree

Important Terminology related to Decision Trees:

1. **Root Node:** It represents the entire population or sample and this further gets divided into two or more homogeneous sets.
2. **Splitting:** It is a process of dividing a node into two or more sub-nodes.
3. **Decision Node:** When a sub-node splits into further sub-nodes, then it is called the decision node.
4. **Leaf / Terminal Node:** Nodes do not split is called Leaf or Terminal node.
5. **Pruning:** When we remove sub-nodes of a decision node, this process is called pruning. You can say the opposite process of splitting.
6. **Branch / Sub-Tree:** A subsection of the entire tree is called branch or sub-tree.
7. **Parent and Child Node:** A node, which is divided into sub-nodes is called a parent node of sub-nodes whereas sub-nodes are the child of a parent node.

NIET
Greater Noida
Autonomous Institute
Estd.2001
Get Future Ready

Decision Tree



Note:- A is parent node of B and C.

Decision Tree

- Decision trees classify the examples by sorting them down the tree from the root to some leaf/terminal node, with the leaf/terminal node providing the classification of the example.

- Each node in the tree acts as a test case for some attribute, and each edge descending from the node corresponds to the possible answers to the test case.

- This process is recursive in nature and is repeated for every subtree rooted at the new node.

Decision Tree

Assumptions while creating Decision Tree:

- In the beginning, the whole training set is considered as the **root.**
- Feature values are preferred to be categorical.
- If the values are continuous then they are discretized prior to building the model.
- Records are **distributed recursively** on the basis of attribute values.
- Order to placing attributes as root or internal node of the tree is done by using some statistical approach.

Decision Tree

How do Decision Trees work?

- The decision of making strategic splits heavily affects a tree's accuracy.
- The decision criteria are different for classification and regression trees.
- Decision trees use multiple algorithms to decide to split a node into two or more sub-nodes.
- The creation of sub-nodes increases the homogeneity of resultant sub-nodes.
- In other words, we can say that the purity of the node increases with respect to the target variable.
- The decision tree splits the nodes on all available variables and then selects the split which results in most homogeneous sub-nodes.

Decision Tree

- The algorithm selection is also based on the type of target variables. Let us look at some algorithms used in Decision Trees:
- **ID3** → (extension of D3)
  C4.5 → (successor of ID3)
  CART → (Classification And Regression Tree)
  CHAID → (Chi-square automatic interaction detection Performs multi-level splits when computing classification trees)
  MARS → (multivariate adaptive regression splines)

Decision Tree

The ID3 (Iterative Dichotomiser 3) algorithm builds decision trees using a top-down greedy search approach through the space of possible branches with no backtracking. A greedy algorithm, as the name suggests, always makes the choice that seems to be the best at that moment.

Steps in ID3 algorithm:

1. It begins with the original set S as the root node.
2. On each iteration of the algorithm, it iterates through the very unused attribute of the set S and calculates **Entropy(H)** and **Information gain(IG)** of this attribute.
3. It then selects the attribute which has the **smallest Entropy** or **Largest Information gain.**
4. The set S is then split by the selected attribute to produce a subset of the data.
5. The algorithm continues to recur on each subset, considering only attributes never selected before.

NIET
Greater Noida
Autonomous Institute
Estd.2001
Get Future Ready

Decision Tree

(1) ID3 Algorithm.

Information Gain of Weather

Day	Weather	Temp	Humidity	wind	Play football?
1	Sunny	Hot	High	weak	No
2	Sunny	Hot	High	Strong	No
3	Cloudy	Hot	High	weak	Yes
4	Rain	Mild	High	weak	Yes
5	Rain	Cool	Normal	weak	Yes
6	Rain	Cool	Normal	Strong	No
7	Cloudy	Cool	Normal	Strong	Yes
8	Sunny	Mild	High	weak	No
9	Sunny	Cool	normal	weak	Yes
10	Rain	Mild	normal	weak	Yes

Decision Tree

- **Attribute Selection Measures**
- If the dataset consists of **N** attributes, then deciding which attribute to place at the root or at different levels of the tree as internal nodes is a complicated step.
- By just randomly selecting any node to be the root can’t solve the issue.
- If we follow a random approach, it may give us bad results with low accuracy.
- For solving this attribute selection problem, researchers worked and devised some solutions.

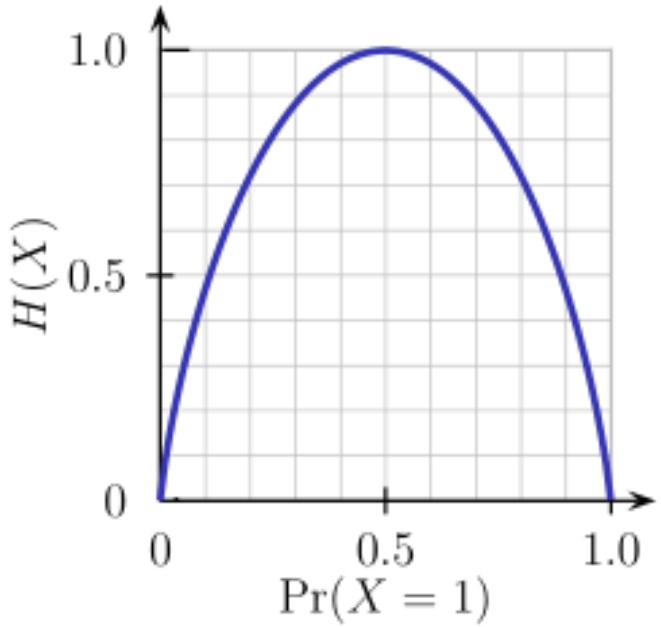
Decision Tree

They suggested using some *criteria* like :

1. **Entropy**,
2. **Information gain,**
3. **Gini index,**
4. **Gain Ratio,**
5. **Reduction in Variance**
6. **Chi-Square**

Decision Tree

- **Entropy:**
- Entropy is a measure of the randomness in the information being processed.
- The higher the entropy, the harder it is to draw any conclusions from that information.
- Flipping a coin is an example of an action that provides information that is random.



Decision Tree

- From the above graph, it is quite evident that the entropy H(X) is zero when the probability is either 0 or 1.
- The Entropy is maximum when the probability is 0.5 because it projects perfect randomness in the data and there is no chance if perfectly determining the outcome.
- **ID3 follows the rule — A branch with an entropy of zero is a leaf node and A branch with entropy more than zero needs further splitting.**

Decision Tree

- Mathematically Entropy for 1 attribute is represented as:

$$E(S) = \sum_{i=1}^{c} - p_i \log_2 p_i$$

Play Golf	
Yes	No
9	5



Entropy(PlayGolf) = Entropy (5,9)
= Entropy (0.36, 0.64)
= - (0.36 $\log_2$ 0.36) - (0.64 $\log_2$ 0.64)
= 0.94

- W **S or**

P

Decision Tree

- Mathematically Entropy for multiple attributes is represented as:

$$E(T, X) = \sum_{c \in X} P(c) E(c)$$

		Play Golf		
		Yes	No	
Outlook	Sunny	3	2	5
	Overcast	4	0	4
	Rainy	2	3	5
				14

where **T→ Current state and X → Selected attribute**

E(PlayGolf, Outlook) = **P**(Sunny)*__E__(3,2) + **P**(Overcast)*__E__(4,0) + **P**(Rainy)*__E__(2,3)

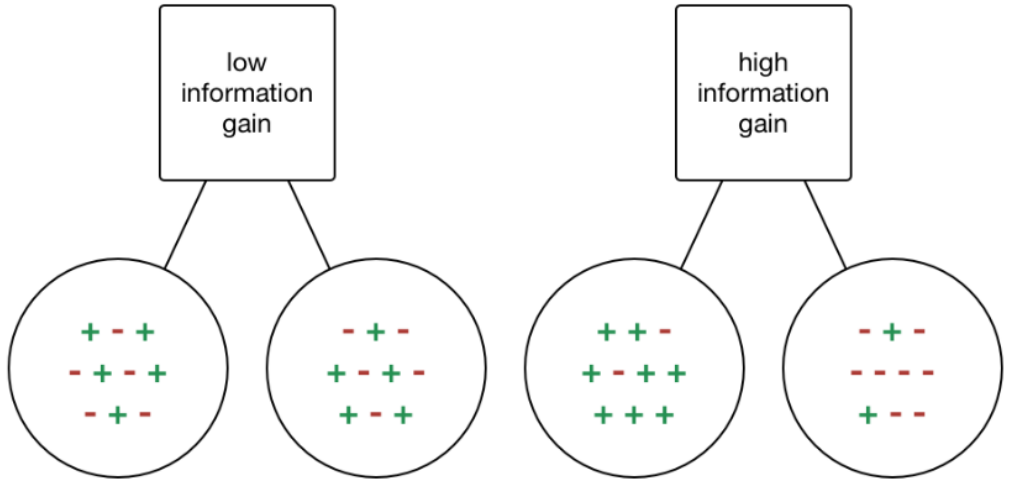
= (5/14)*0.971 + (4/14)*0.0 + (5/14)*0.971

= 0.693

Decision Tree

Information Gain:

- Information gain or **IG** is a statistical property that measures how well a given attribute separates the training examples according to their target classification.
- Constructing a decision tree is all about finding an attribute that returns the highest information gain and the smallest entropy.



Decision Tree

- Information gain is a decrease in entropy.
- It computes the difference between entropy before split and average entropy after split of the dataset based on given attribute values.
- ID3 (Iterative Dichotomiser) decision tree algorithm uses information gain.

Decision Tree

- Mathematically, IG is represented as:

$$\text{Information Gain}(T,X) = \text{Entropy}(T) - \text{Entropy}(T, X)$$

IG(PlayGolf, Outlook) = E(PlayGolf) - E(PlayGolf, Outlook)
= 0.940 - 0.693
= 0.247

$$Information\ Gain = Entropy(before) - \sum_{j=1}^{K} Entropy(j,\ after)$$

Where “before” is the dataset before the split, K is the number of subsets generated by the split, and (j, after) is subset j after the split.

Decision Tree

Gini Index

- You can understand the Gini index as a cost function used to evaluate splits in the dataset.
- It is calculated by subtracting the sum of the squared probabilities of each class from one.
- It favors larger partitions and easy to implement whereas information gain favors smaller partitions with distinct values.

$$Gini = 1 - \sum_{i=1}^{C} (p_i)^2$$

- Gini Index works with the categorical target variable "Success" or "Failure". It performs only Binary splits.
- Higher the value of Gini index higher the homogeneity.

Decision Tree

- **Steps to Calculate Gini index for a split**

1. Calculate Gini for sub-nodes, using the above formula for success(p) and failure(q) (p^2+q^2).
2. Calculate the Gini index for split using the weighted Gini score of each node of that split.

- CART (Classification and Regression Tree) uses the Gini index method to create split points.

Decision Tree

Gain ratio

- Information gain is biased towards choosing attributes with a large number of values as root nodes.
- It means it prefers the attribute with a large number of distinct values.
- C4.5, an improvement of ID3, uses Gain ratio which is a modification of Information gain that reduces its bias and is usually the best option.
- Gain ratio overcomes the problem with information gain by taking into account the number of branches that would result before making the split.
- It corrects information gain by taking the intrinsic information of a split into account.

$$Gain\ Ratio = \frac{Information\ Gain}{SplitInfo} = \frac{Entropy\ (before) - \sum_{j=1}^{K} Entropy(j, after)}{\sum_{j=1}^{K} w_j \log_2 w_j}$$

Where “before” is the dataset before the split, K is the number of subsets generated by the split, and (j, after) is subset j after the split.

Decision Tree

Issues in Decision Tree:

- The common problem with Decision trees, especially having a table full of columns, they fit a lot.
- Sometimes it looks like the tree memorized the training data set.
- If there is no limit set on a decision tree, it will give you 100% accuracy on the training data set because in the worse case it will end up making 1 leaf for each observation.
- Thus this affects the accuracy when predicting samples that are not part of the training set.
- Here are two ways to remove overfitting:
  1. Pruning Decision Trees.
  2. Random Forest

Decision Tree

Pruning Decision Trees:

- In **pruning**, you trim off the branches of the tree, i.e., remove the decision nodes starting from the leaf node such that the overall accuracy is not disturbed.
- This is done by segregating the actual training set into two sets: training data set, D and validation data set, V.
- Prepare the decision tree using the segregated training data set, D.
- Then continue trimming the tree accordingly to optimize the accuracy of the validation data set, V.

NIET
Greater Noida
Autonomous Institute
Estd.2001
Get Future Ready

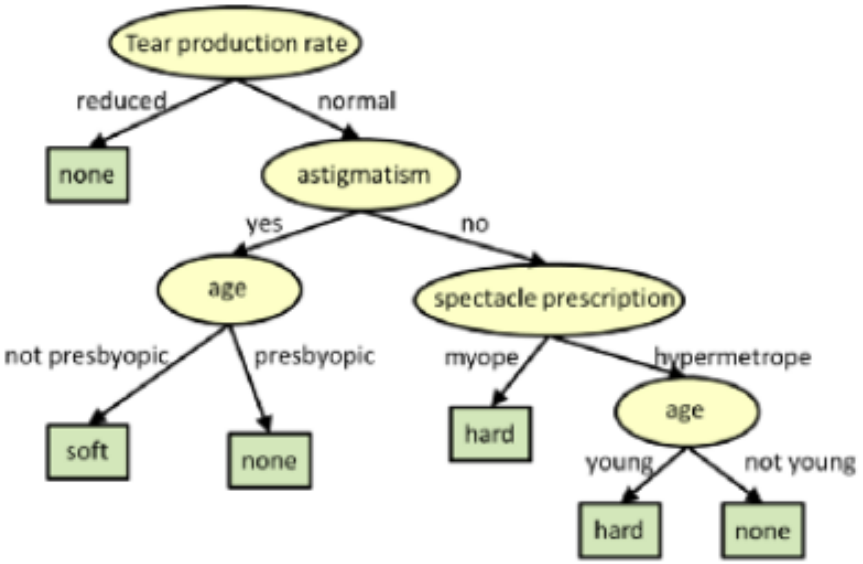
Decision Tree

Question

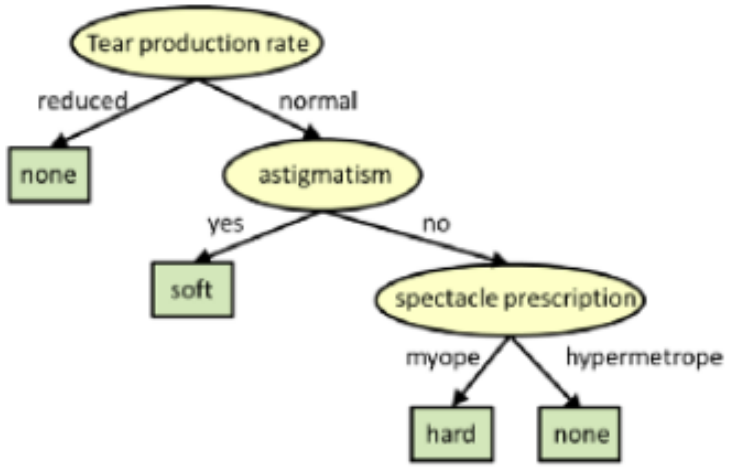
For the following Medical Diagnosis Data, Create Decision Tree.

S.No	Sore Throat	Fever	Swollen Glands	Congestion	Headache	Diagnosis
1	Yes	Y	Y	Y	Y	Strep Throat
2	No	N	N	Y	Y	Allergy
3	Yes	Y	N	Y	N	Cold
4	Yes	N	Y	N	N	Strep Throat
5	No	Y	N	Y	N	Cold
6	No	N	N	Y	N	Allergy
7	No	N	Y	N	N	Strep Throat
8	Yes	N	N	Y	Y	Allergy
9	No	Y	N	Y	Y	Cold
10	Yes	Y	N	Y	Y	Cold.

Decision Tree



Original Tree



Pruned Tree

In the above diagram, the 'Age' attribute in the left-hand side of the tree has been pruned as it has more importance on the right-hand side of the tree, hence removing overfitting.

C 4.5

C4.5

C4.5 decision tree is a modification over the ID3 Decision Tree. C4.5 uses the Gain Ratio as the *goodness function* to split the dataset, unlike ID3 which used the Information Gain.

The Information Gain function tends to prefer the features with more categories as they tend to have lower entropy. This results in overfitting of the training data. Gain Ratio mitigates this issue by penalising features for having a more categories using a formula called Split Information or Intrinsic Information.

Consider the calculation of Split Information for the Outlook feature.

C 4.5

Pseudocode of C4.5 algorithm

- Let's see the Pseudocode of the C4.5 algorithm in data mining.

1. First, notice the base
2. For each attribute X, find the **normalized information gain ratio** by splitting between X.
3. Suppose that X is an attribute with the highest **normalized information gain ratio**.
4. Create a decision node that splits on attribute X.
5. Repeat it on the sublists obtained by splitting the attribute X, and add these nodes as children of the node.

Advantages of C4.5 over other Decision Tree systems

1. The algorithm is very helpful in Mitigating the overfitting because C4.5 inherently employs the Single Pass Pruning Process.
2. C4.5 can work with Discrete data and can also work with Continuous Data
3. C4.5 is very helpful in solving the issues of data incompleteness

C 4.5

Outlook	Temp	Humidity	Windy	Play
Sunny	Hot	High	False	No
Sunny	Hot	High	True	No
Overcast	Hot	High	False	Yes
Rainy	Mild	High	False	Yes
Rainy	Cool	Normal	False	Yes
Rainy	Cool	Normal	True	No
Overcast	Cool	Normal	True	Yes
Sunny	Mild	High	False	No
Sunny	Cool	Normal	False	Yes
Rainy	Mild	Normal	False	Yes
Sunny	Mild	Normal	True	Yes
Overcast	Mild	High	True	Yes
Overcast	Hot	Normal	False	Yes
Rainy	Mild	High	True	No

C 4.5

- ID3 uses pure Information Gain (biased to many-valued attributes).
- **C4.5 uses Gain Ratio** to correct that bias (penalizes splits that fragment the data into many tiny subsets).
- If two attributes have very close GainRatios, prefer the simpler / more interpretable attribute or use tie-breaking heuristics (e.g., higher Gain if splitinfo small).
- For continuous attributes C4.5 searches thresholds and treats them as binary splits (<= threshold / > threshold).
- When SplitInfo is extremely small (near 0), avoid dividing by it — treat that attribute cautiously.

C 4.5

Now Applying the value of Split Information to the Gain Ratio:

$$\text{GainRatio}(A) = \frac{\text{Gain}(A)}{\text{SplitInfo}(A)}$$

Formula to compute the Gain Ratio

Decision Tree using CART algorithm Solved Example

Outlook	Temp	Humidity	Windy	Play
Sunny	Hot	High	False	No
Sunny	Hot	High	True	No
Overcast	Hot	High	False	Yes
Rainy	Mild	High	False	Yes
Rainy	Cool	Normal	False	Yes
Rainy	Cool	Normal	True	No
Overcast	Cool	Normal	True	Yes
Sunny	Mild	High	False	No
Sunny	Cool	Normal	False	Yes
Rainy	Mild	Normal	False	Yes
Sunny	Mild	Normal	True	Yes
Overcast	Mild	High	True	Yes
Overcast	Hot	Normal	False	Yes
Rainy	Mild	High	True	No

Decision Tree using CART algorithm Solved Example

Temp

Hot	4	Yes	2
		No	2
Mild	6	Yes	4
		No	2
Cold	4	Yes	3
		No	1

Attribute	**Rules**	**Error**	**Total Error**
Temp	Hot → No	2/4	**5/14**
	Mild → Yes	2/6	
	Cool → Yes	1/4	

Decision Tree using CART algorithm Solved Example

Humidity

High	7	Yes	3
		No	4
Normal	7	Yes	6
		No	1

Rules, individual error, and total for **Humidity** attribute

Attribute	**Rules**	**Error**	**Total Error**
Humidity	High→ No	3/7	**4/14**
	Normal → Yes	1/7	

Decision Tree using CART algorithm Solved Example

Windy

False	8	Yes	6
		No	2
True	6	Yes	3
		No	3

Rules, individual error, and total for **Humidity** attribute

Attribute	Rules	Error	Total Error
Windy	True → No	3/6	**5/14**
	False → Yes	2/8	

Decision Tree using CART algorithm Solved Example

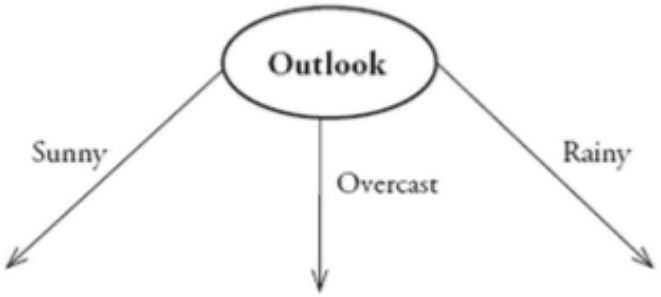
Attribute	Rules	Error	Total Error
Outlook	Sunny → No	2/5	**4/14**
	Overcast → Yes	0/4	
	Rainy → Yes	2/5	
Temp	hot → No	2/4	**5/14**
	Mild → Yes	2/6	
	Cool → Yes	1/4	
Humidity	high → No	3/7	**4/14**
	Normal → Yes	1/7	
Windy	False → Yes	2/8	**5/14**
	True → No	3/6	

Decision Tree using CART algorithm Solved Example

From the above table, we can notice that the attributes **Outlook** and **Humidity** have the same minimum error that is 4/14. Hence we consider the individual attribute value errors. The outlook attribute has one rule which generates zero error that is the rule **Overcast** → **Yes**. Hence we consider the Outlook as the splitting attribute.

Now we build the tree with Outlook as the root node. It has three branches for each possible value of the outlook attribute. As the rule, **Overcast** → **Yes** generates zero error. When the outlook attribute value is overcast we get the result as **Yes**. For the remaining two attribute values we consider the subset of data and continue building the tree. Tree with Outlook as root node is,

Decision Tree using CART algorithm Solved Example



Temp	Humidity	Windy	Play
Hot	High	False	No
Hot	High	True	No
Mild	High	False	No
Cool	Normal	False	Yes
Mild	Normal	True	Yes

YES

Temp	Humidity	Windy	Play
Mild	High	False	Yes
Cool	Normal	False	Yes
Cool	Normal	True	No
Mild	Normal	False	Yes
Mild	High	True	No

Decision Tree using CART algorithm Solved Example

Left subtree,

Temp	Humidity	Windy	Play
Hot	High	False	No
Hot	High	True	No
Mild	High	False	No
Cool	Normal	False	Yes
Mild	Normal	True	Yes

Consolidated rules, errors for individual attributes values, and total error of the attribute are given below.

Attribute	Rules	Error	Total Error
Temp	Hot → No	0/2	1/5
	Mild → No	1/2	
	Cool → Yes	0/1	
Humidity	High → No	0/3	0/5
	Normal → Yes	0/2	
Windy	False → No	1/3	2/5
	True → Yes	1/2	

From the above table, we can notice that Humidity has the lowest error. Hence Humidity is considered as the splitting attribute. Also, when Humidity is High the answer is No as it produces zero errors. Similarly, when Humidity is Normal the answer is Yes, as it produces zero errors.

Decision Tree using CART algorithm Solved Example

Right subtree,

Temp	Humidity	Windy	Play
Mild	High	False	Yes
Cool	Normal	False	Yes
Cool	Normal	True	No
Mild	Normal	False	Yes
Mild	High	True	No

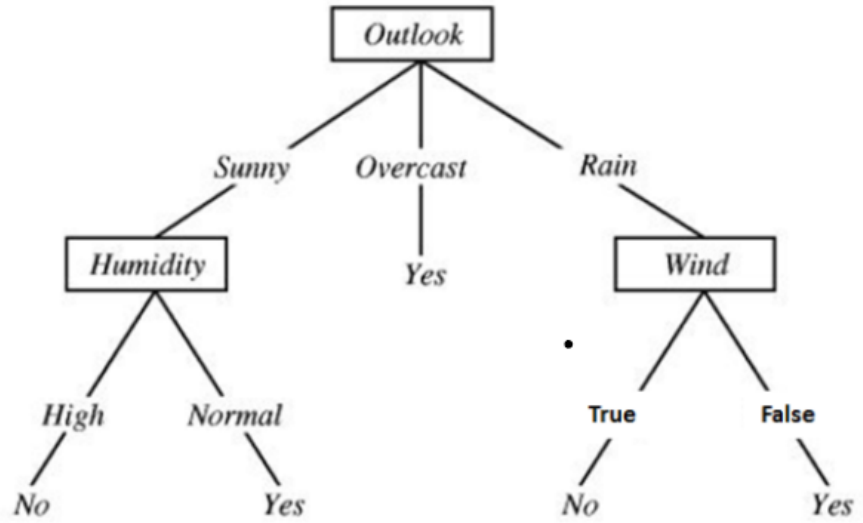
Consolidated rules, errors for individual attributes values, and total error of the attribute are given below.

Attribute	Rules	Error	Total Error
Temp	Mild → Yes	1/3	2/5
	Cool → yes	1/2	
Humidity	High → No	1/2	2/5
	Normal → Yes	1/3	
Windy	False → Yes	0/3	0/5
	True → No	0/2	

From the above table, we can notice that Windyhas the lowest error. Hence Windy is considered as the splitting attribute. Also, when Windy is False the answer is Yes as it produces zero errors. Similarly, when Windy is True the answer is No, as it produces zero errors.

Decision Tree using CART algorithm Solved Example

The final decision tree for the given Paly Tennis data set is,



Also, from the above decision tree the prediction for the new example:

Outlook	Temp	Humidity	Windy	Play
Sunny	Hot	Normal	True	?

is, **Yes**

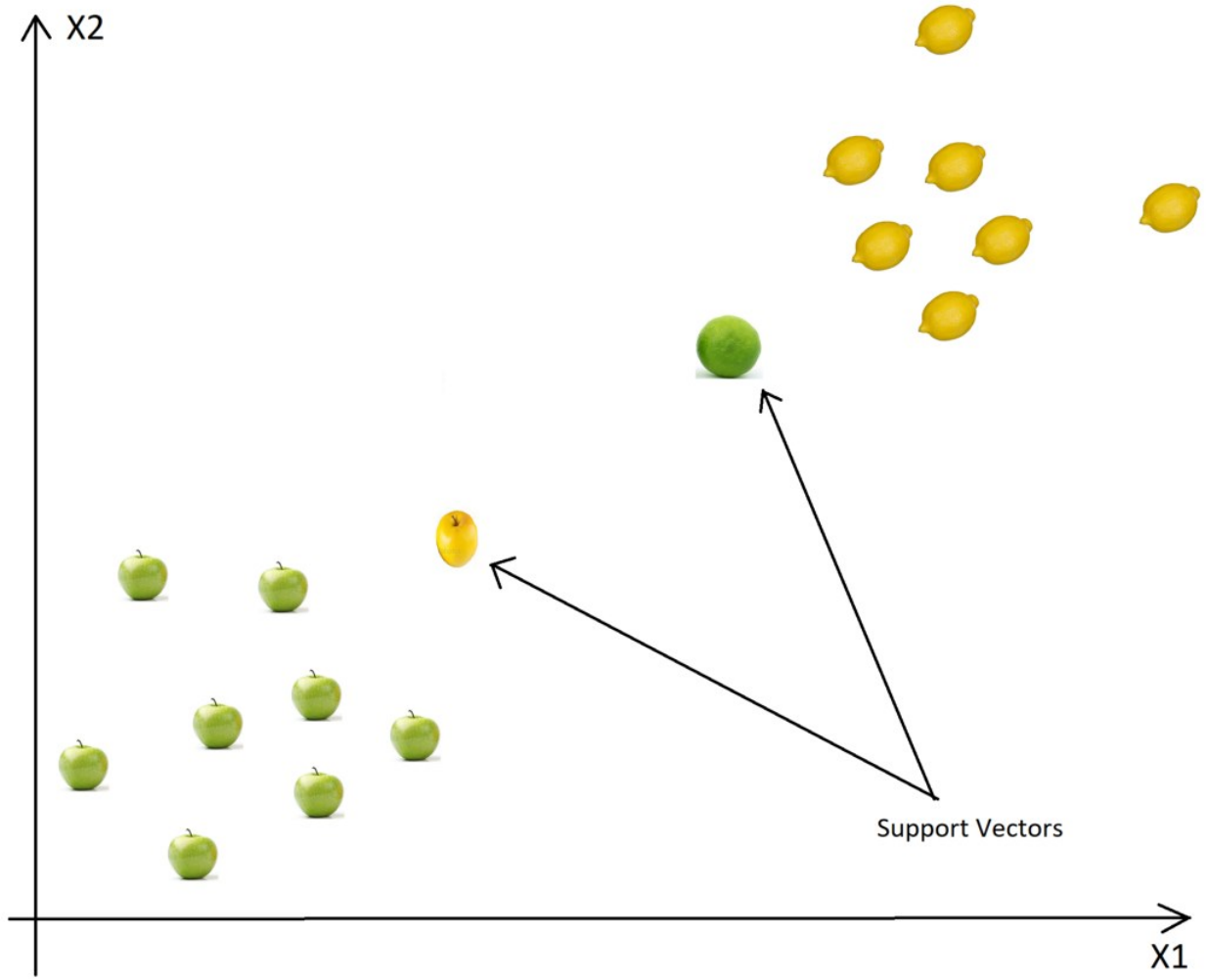
SVM

- The **Support Vector Machine** is a **supervised learning algorithm** mostly used for **classification** but it can be used also for **regression**.
- The main idea is that based on the labeled data (training data) the algorithm tries to find the **optimal hyperplane** which can be used to classify new data points.
- In two dimensions the hyperplane is a simple line.
- **Usually** a learning algorithm tries to learn the **most common characteristics (what differentiates one class from another)** of a class and the classification is based on those representative characteristics learnt (so classification is based on differences between classes).
- The **SVM** works in the other way around. It **finds** the **most similar examples** between classes. Those will be the **support vectors**.

SVM

- As an example, lets consider two classes, apples and lemons.
- Other algorithms will learn the most evident, most representative characteristics of apples and lemons, like apples are green and rounded while lemons are yellow and have elliptic form.
- In contrast, SVM will search for apples that are very similar to lemons, for example apples which are yellow and have elliptic form.
- This will be a support vector. The other support vector will be a lemon similar to an apple (green and rounded).
- So **other algorithms** learns the **differences** while **SVM** learns **similarities**.

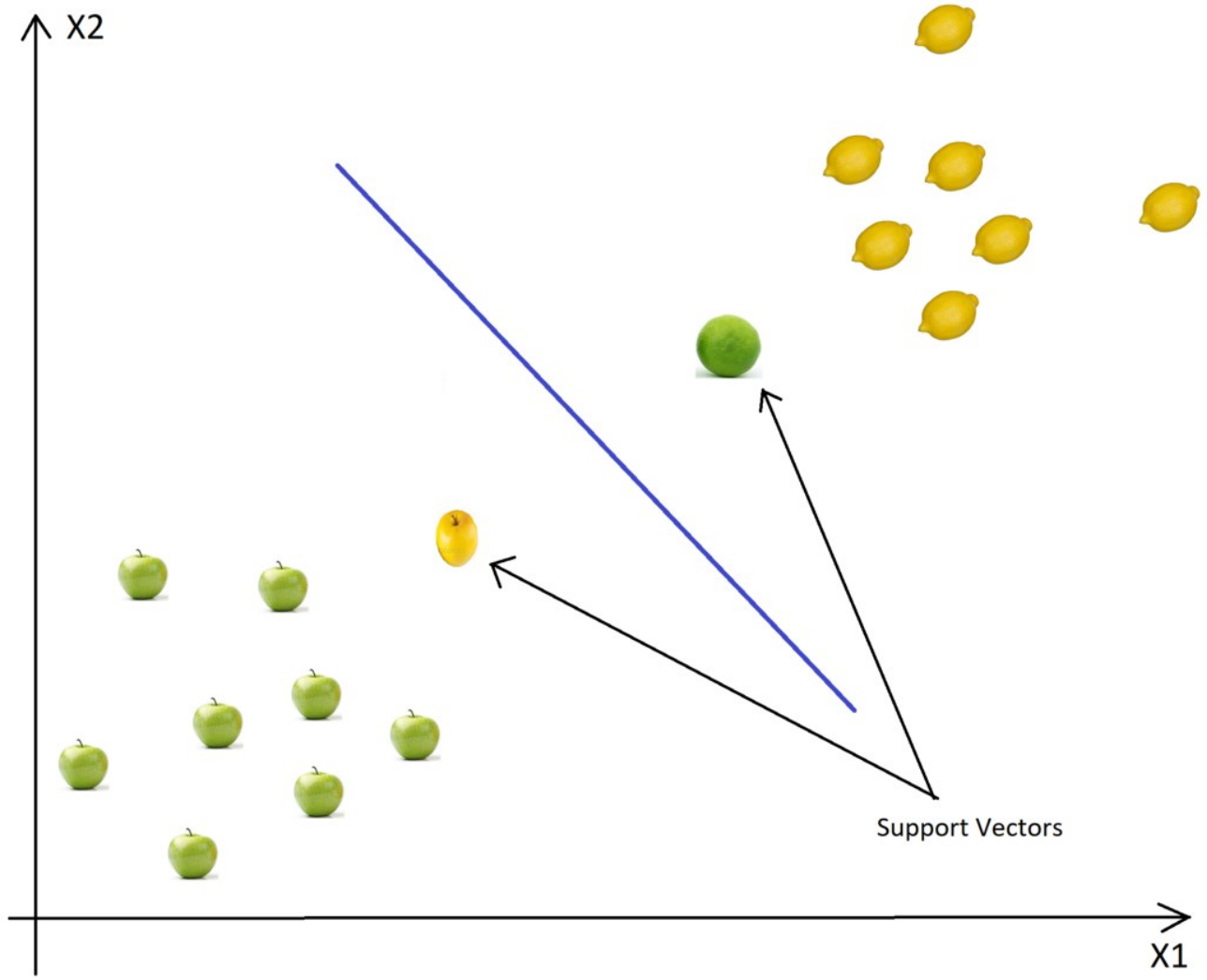
SVM



SVM

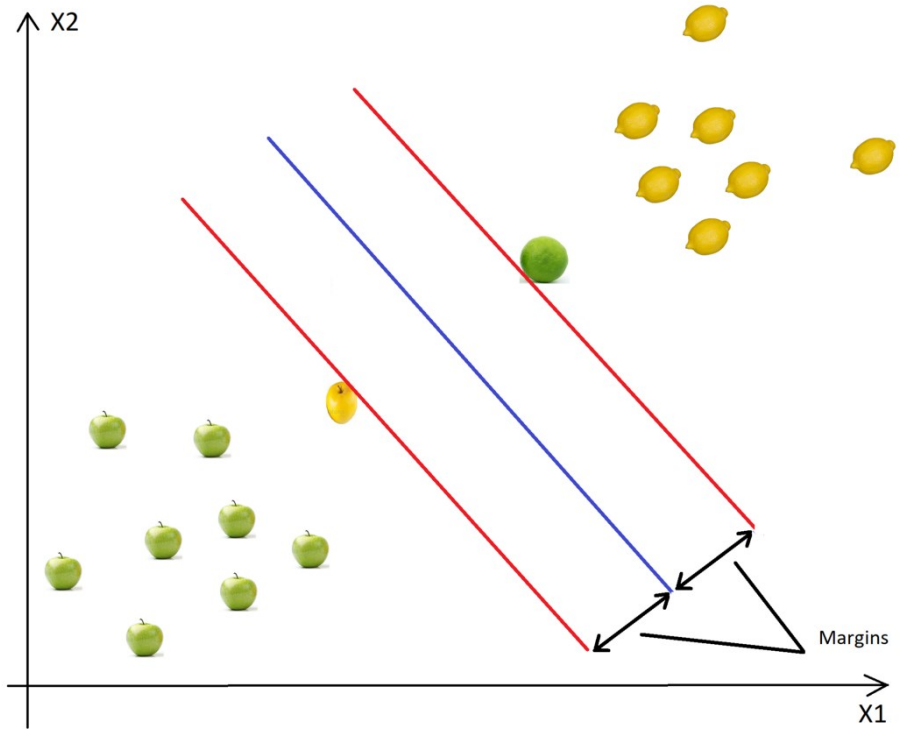
- As we go from left to right, all the examples will be classified as apples until we reach the yellow apple.
- From this point, the confidence that a new example is an apple drops while the lemon class confidence increases.
- When the lemon class confidence becomes greater than the apple class confidence, the new examples will be classified as lemons (somewhere between the yellow apple and the green lemon).
- Based on these support vectors, the algorithm tries to find **the best hyperplane that separates the classes**.
- In 2D the hyperplane is a line, so it would look like this:

SVM



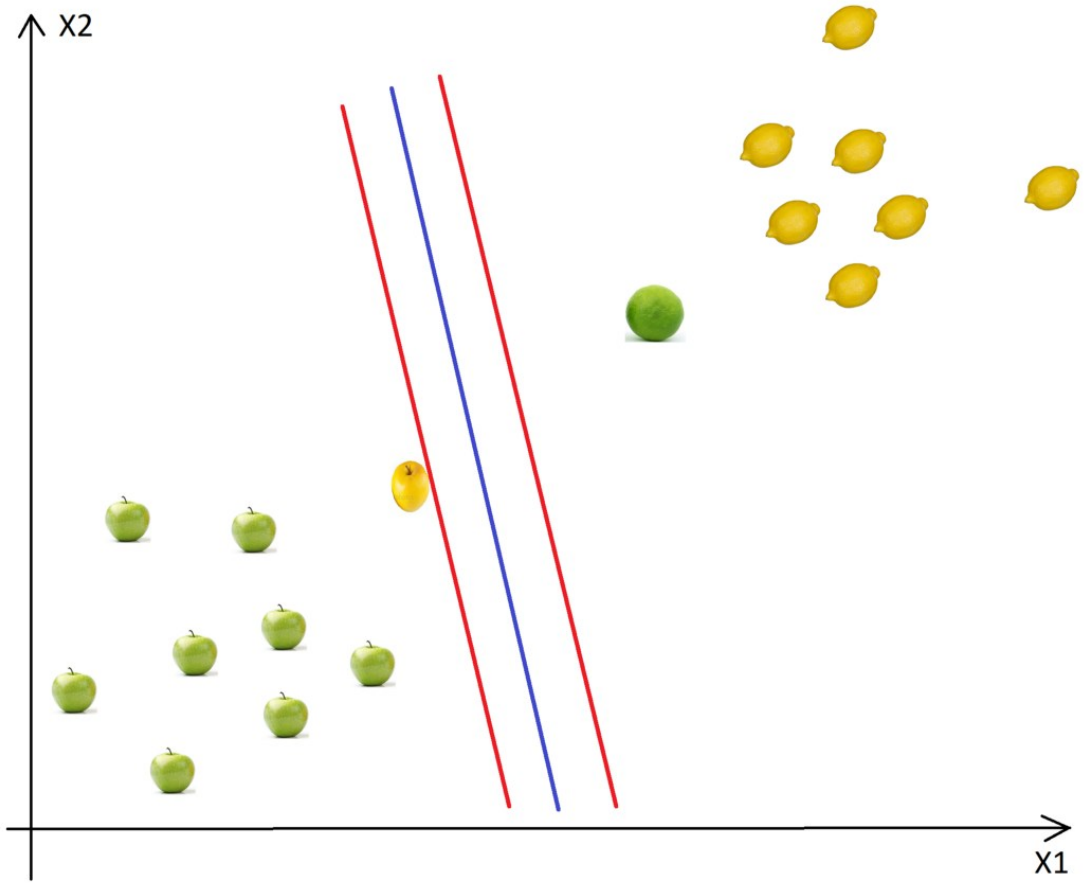
SVM

- **Finding the Optimal Hyperplane**
- Intuitively the **best line** is the line that is **far away from both apple and lemon examples** (has the largest margin). To have optimal solution, we have to **maximize the margin in both ways** (if we have multiple classes, then we have to maximize it considering each of the classes).



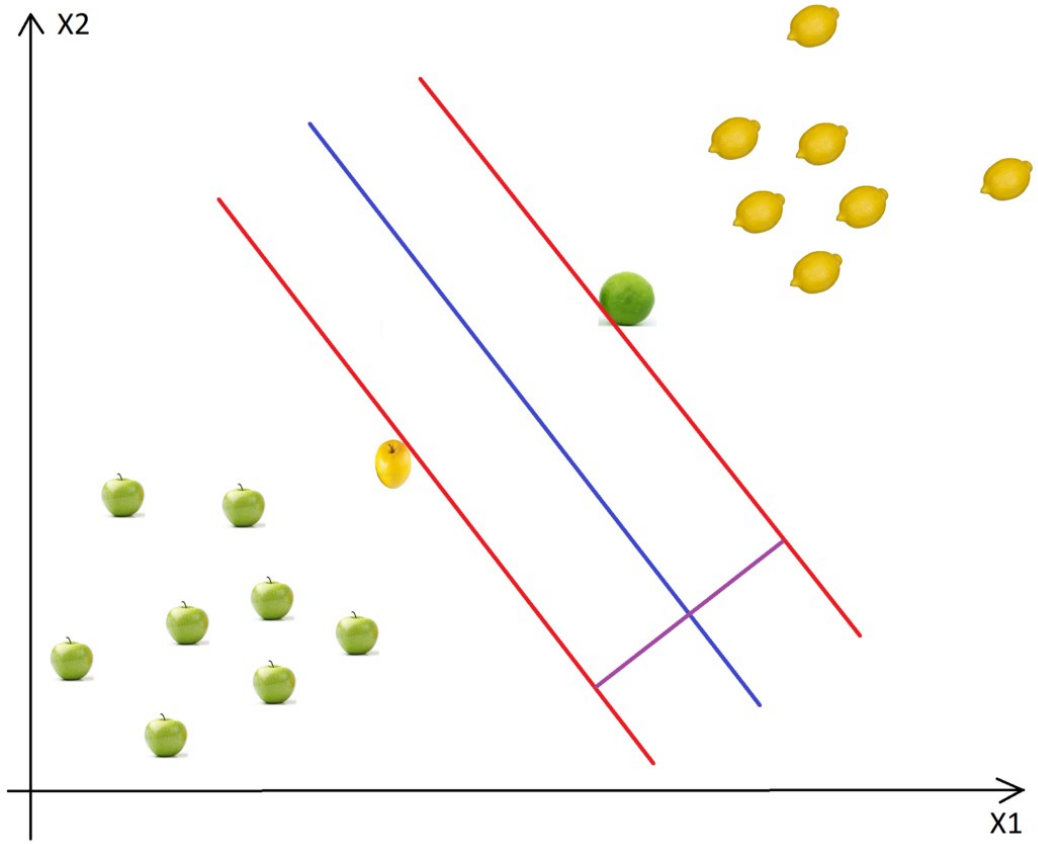
SVM

- So if we compare the picture above with the picture below, we can easily observe, that the first is the optimal hyperplane (line) and the second is a sub-optimal solution, because the margin is far shorter.



SVM

- Because we want to maximize the margins taking in consideration **all the classes**, instead of using one margin for each class, **we use a "global" margin**, **which takes in consideration all the classes**. This margin would look like the purple line in the following picture:



SVM

- This margin is **orthogonal** to the boundary and **equidistant** to the support vectors.
- So where do we have vectors? Each of the calculations (calculate distance and optimal hyperplanes) are made in **vectorial space**, so each data point is considered a vector. The **dimension** of the space **is defined by the number of attributes** of the examples. To understand the math behind, please read this brief mathematical description of vectors, hyperplanes and optimizations: SVM Succintly.
- All in all, **support vectors** are data points that **defines the position and the margin of the hyperplane**. We call them **"support" vectors**, because these are the representative data points of the classes, **if we move one of them, the position and/or the margin will change**. Moving other data points won't have effect over the margin or the position of the hyperplane.
- To make classifications, we don't need all the training data points (like in the case of KNN), we have to save only the support vectors. In worst case all the points will be support vectors, but this is very rare and if it happens, then you should check your model for errors or bugs.
- So basically the **learning is equivalent with finding the hyperplane with the best margin**, so it is a simple **optimization problem**.

SVM

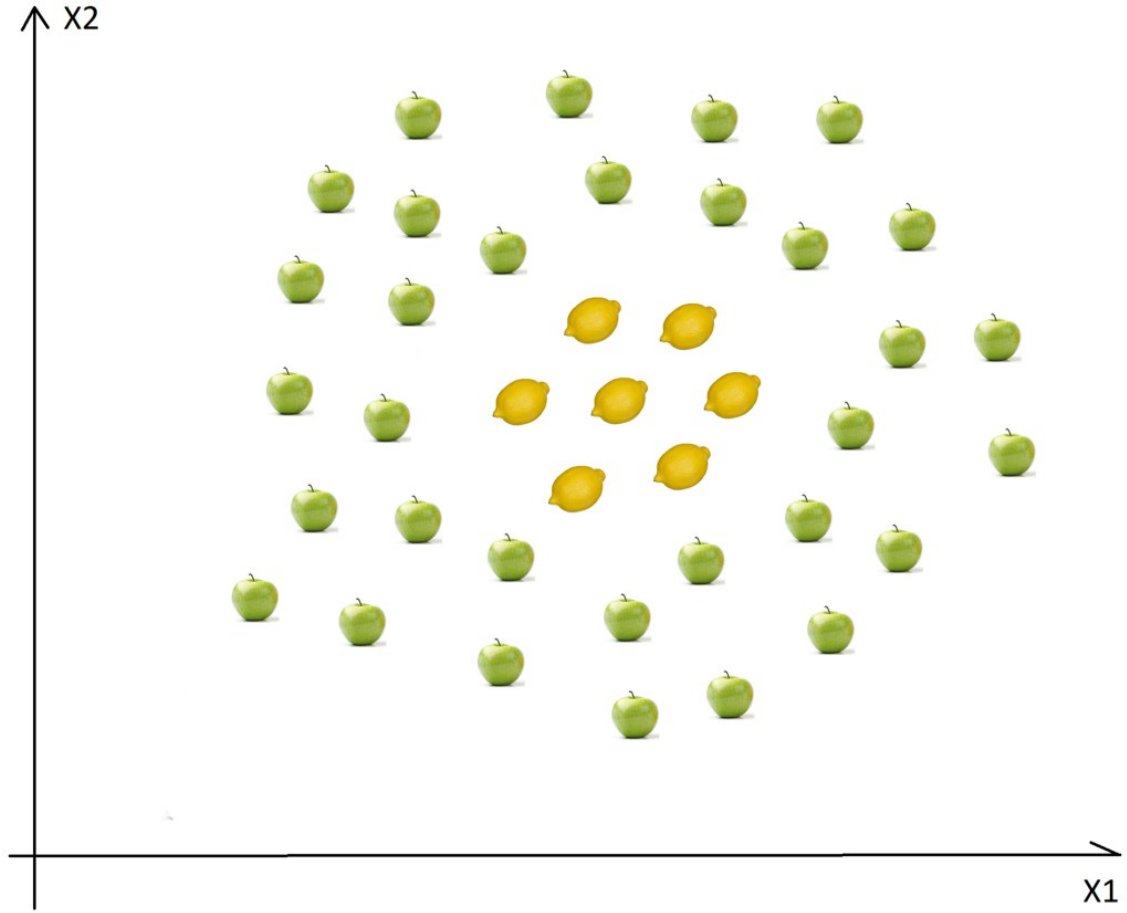
- **Basic Steps**
- The basic steps of the SVM are:

1. select **two hyperplanes** (in 2D) which separates the data **with no points between them** (red lines)
2. **maximize their distance** (the margin)
3. the **average line** (here the line half way between the two red lines) will be the **decision boundary**

- This is very nice and easy, but finding the best margin, the optimization problem is not trivial (it is easy in 2D, when we have only two attributes, but what if we have N dimensions with N a very big number)
- To solve the optimization problem, we use the **Lagrange Multipliers**. To understand this technique you can read the following two articles: Duality Langrange Multiplier and A Simple Explanation of Why Langrange Multipliers Wroks.
- Until now we had linearly separable data, so we could use a line as class boundary. But what if we have to deal with non-linear data sets?

SVM

- **SVM for Non-Linear Data Sets**
- An example of non-linear data is:

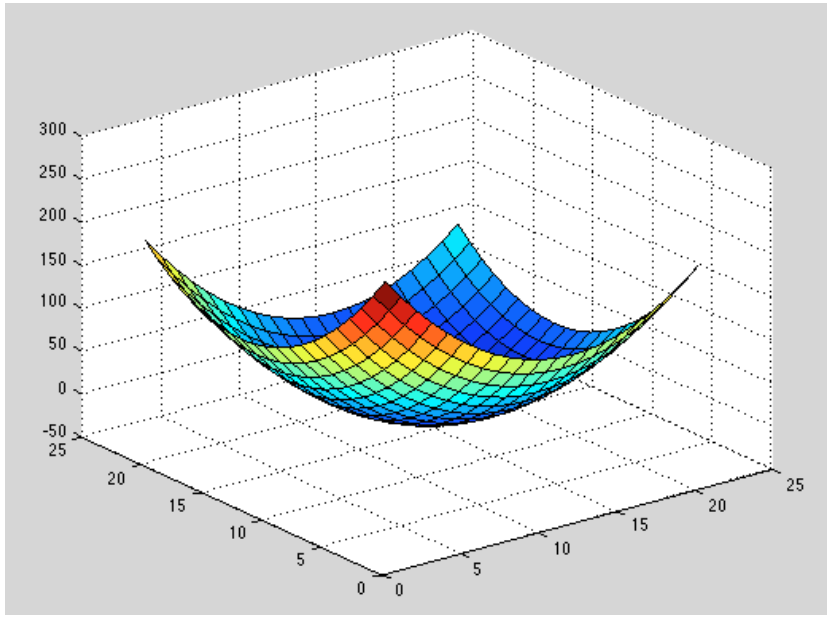


SVM

- In this case **we cannot find a straight line** to separate apples from lemons. So how can we solve this problem. We will use the **Kernel Trick!**
- The basic idea is that when a data set is inseparable in the current dimensions, **add another dimension**, maybe that way the data will be separable.
- Just think about it, the example above is in 2D and it is inseparable, but maybe in 3D there is a gap between the apples and the lemons, maybe there is a level difference, so lemons are on level one and lemons are on level two.
- In this case we can easily draw a separating hyperplane (in 3D a hyperplane is a plane) between level 1 and 2.

SVM

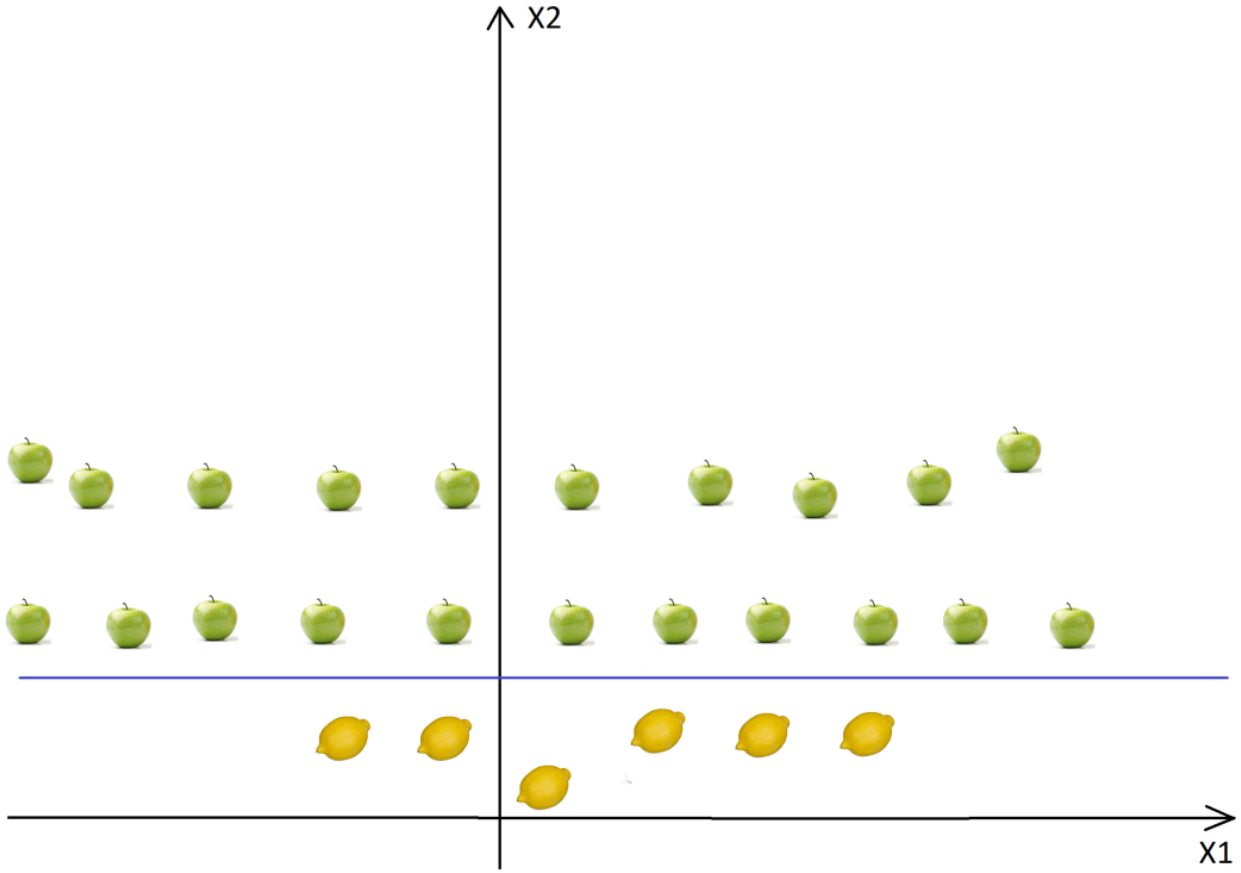
- **Mapping to Higher Dimensions**
- To solve this problem we **shouldn't just blindly add another dimension**, we should transform the space so we generate this level difference intentionally.
- **Mapping from 2D to 3D**
- Let's assume that we add another dimension called **X3**. Another important transformation is that in the new dimension the points are organized using this formula $\mathbf{x1^2 + x2^2}$.
- If we plot the plane defined by the $\mathbf{x^2 + y^2}$ formula, we will get something like this:



SVM

- Now we have to map the apples and lemons (which are just simple points) to this new space.
- Think about it carefully, what did we do?
- We just used a transformation in which **we added levels based on distance**.
- If you are in the origin, then the points will be on the lowest level.
- As we move away from the origin, it means that we are **climbing the hill** (moving from the center of the plane towards the margins) so the level of the points will be higher.
- Now if we consider that the origin is the lemon from the center, we will have something like this:

SVM



SVM

- Now we can easily separate the two classes.
- These transformations are called **kernels**.
- Popular kernels are:
  - **Polynomial Kernel,**
  - **Gaussian Kernel,**
  - **Radial Basis Function (RBF),**
  - **Laplace RBF Kernel,**
  - **Sigmoid Kernel,**
  - **Anove RBF Kernel**, etc

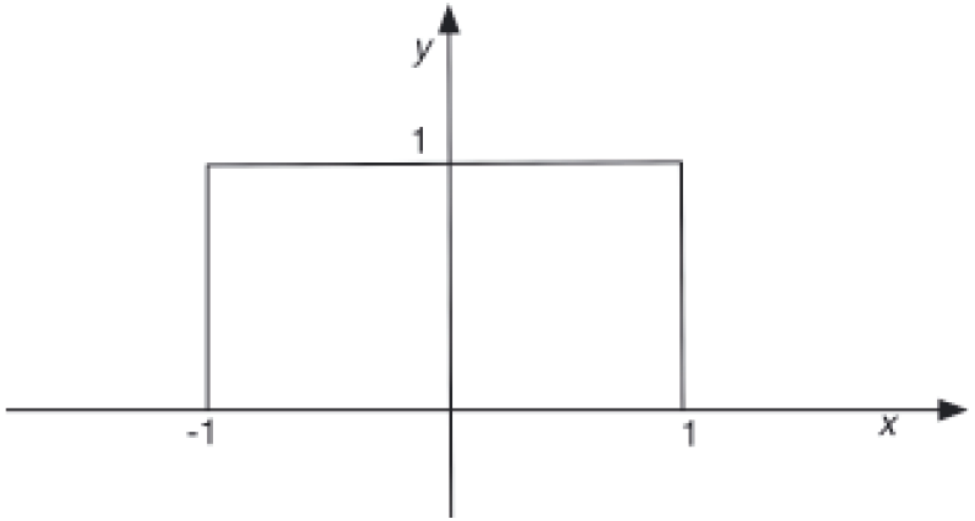
SVM

- **SVM Kernel Functions**
- SVM algorithms use a set of mathematical functions that are defined as the kernel. The function of kernel is to take data as input and transform it into the required form. Different SVM algorithms use different types of kernel functions. These functions can be different types. For example ***linear, nonlinear, polynomial, radial basis function (RBF), and sigmoid.*** Introduce Kernel functions for sequence data, graphs, text, images, as well as vectors. The most used type of kernel function is **RBF.** Because it has localized and finite response along the entire x-axis. The kernel functions return the inner product between two points in a suitable feature space. Thus by defining a notion of similarity, with little computational cost even in very high-dimensional spaces.

SVM

- **Kernel Rules**
- Define kernel or a window function as follows:

$$K\left(\overline{x}\right) = \begin{matrix} 1 & \text{if} \left\|\overline{x}\right\| \leq 1 \\ 0 & \text{otherwise} \end{matrix}$$



SVM

Examples of SVM Kernels

Let us see some common kernels used with SVMs and their uses:

1. Polynomial kernel

- It is popular in image processing.
  Equation is: $k(\mathbf{x_i}, \mathbf{x_j}) = (\mathbf{x_i} \cdot \mathbf{x_j} + 1)^d$

2. Gaussian kernel

- **It is a general-purpose kernel; used when there is no prior knowledge about the data. Equation is:**

$$k(x, y) = \exp\left(-\frac{\|x - y\|^2}{2\sigma^2}\right)$$

SVM

3. Gaussian radial basis function (RBF)

- It is a general-purpose kernel; used when there is no prior knowledge about the data.
  Equation is:

$$k(\mathbf{x_i}, \mathbf{x_j}) = \exp(-\gamma \|\mathbf{x_i} - \mathbf{x_j}\|^2)$$

$$\gamma > 0 \qquad \gamma = 1/2\sigma^2$$

4. Laplace RBF kernel

- It is general-purpose kernel; used when there is no prior knowledge about the data.
  Equation is:

$$k(x, y) = \exp\left(-\frac{\|x - y\|}{\sigma}\right)$$

5. Hyperbolic tangent kernel

- **We can use it in neural networks.**
  Equation is:

$$k(\mathbf{x_i}, \mathbf{x_j}) = \tanh(\kappa \mathbf{x_i} \cdot \mathbf{x_j} + c)$$

SVM

6. Sigmoid kernel

- We can use it as the proxy for neural networks. Equation is

$$k(x, y) = \tanh(\alpha x^T y + c)$$

7. Bessel function of the first kind Kernel

$$k(x, y) = \frac{J_{v+1}(\sigma \|x - y\|)}{\|x - y\|^{-n(v+1)}}$$

8. ANOVA radial basis kernel

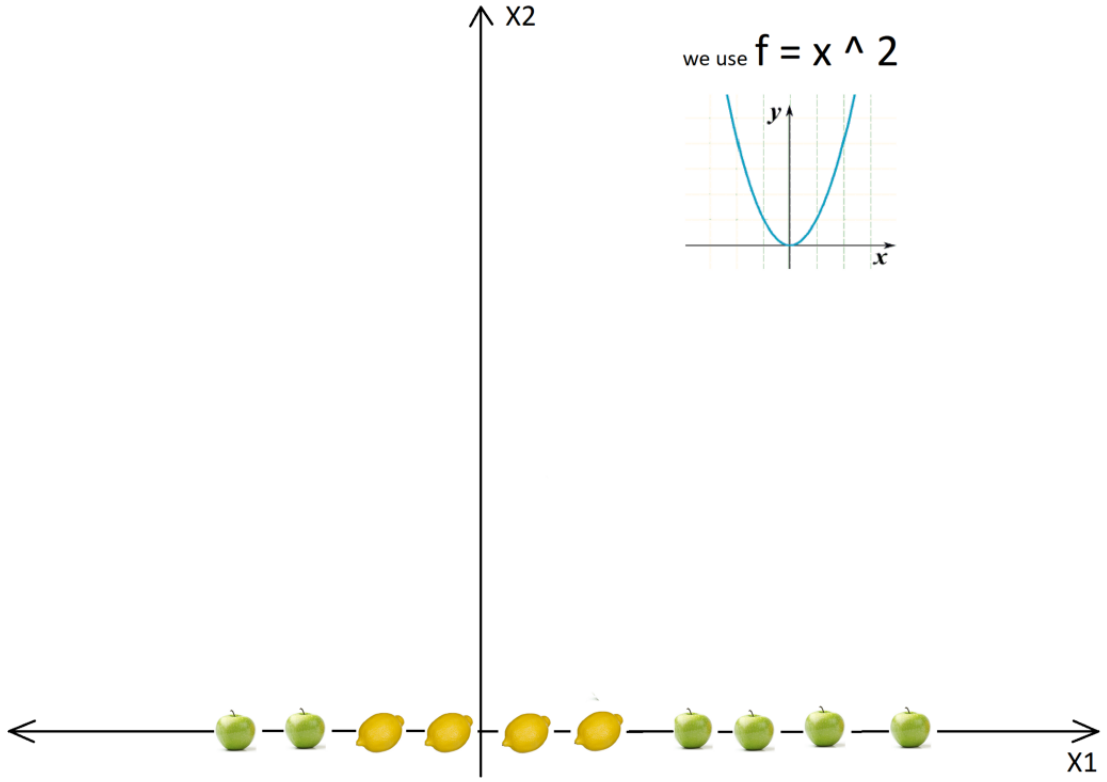
$$k(x, y) = \sum_{k=1}^{n} \exp(-\sigma(x^k - y^k)^2)^d$$

9. Linear splines kernel in one-dimension

$$k(x, y) = 1 + xy + xy\ min(x, y) - \frac{x + y}{2}\ min(x, y)^2 + \frac{1}{3}\min(x, y)^3$$

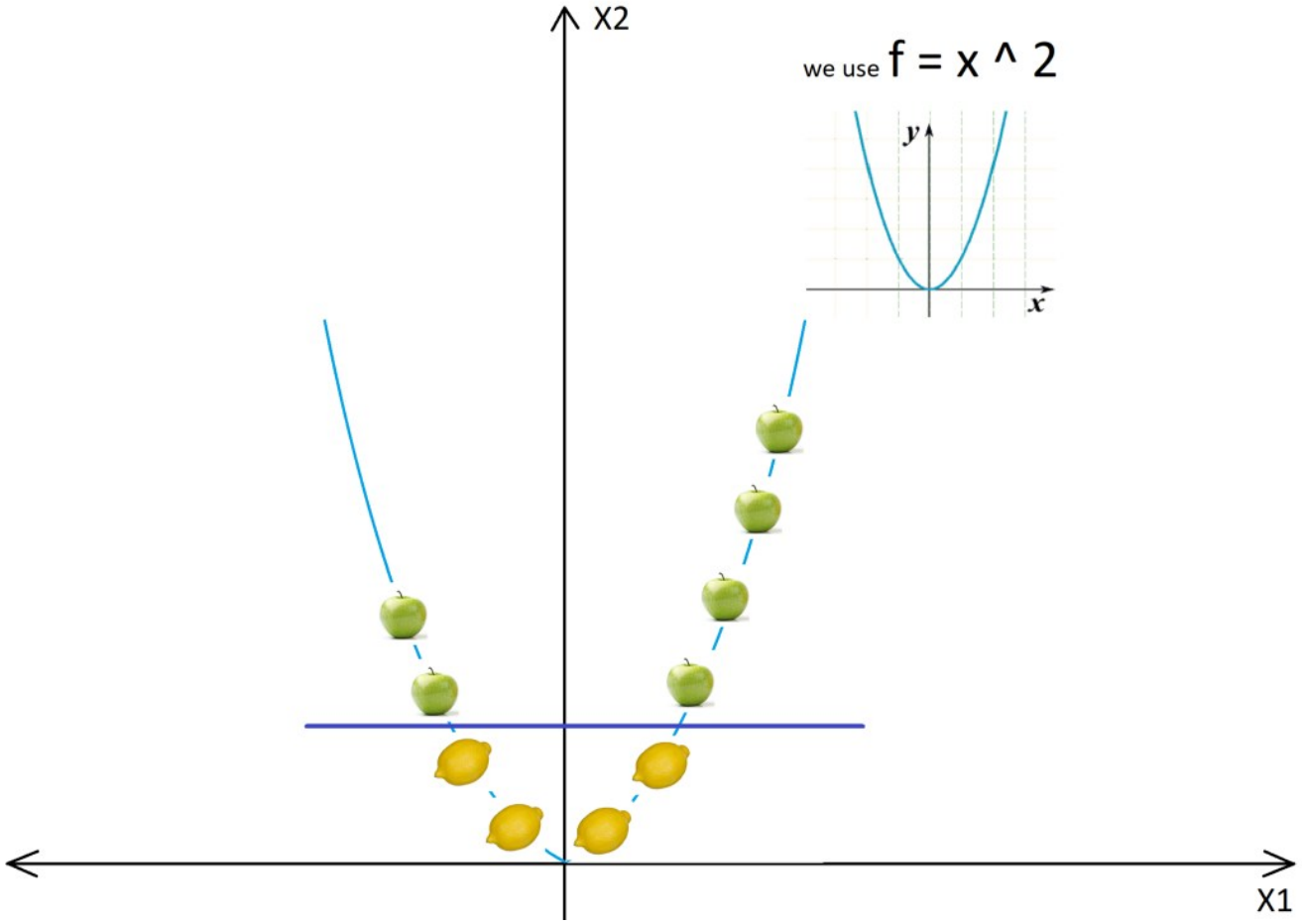
SVM

- **Mapping from 1D to 2D**
- Another, easier example in 2D would be:



SVM

- After using the kernel and after all the transformations we will get:



SVM

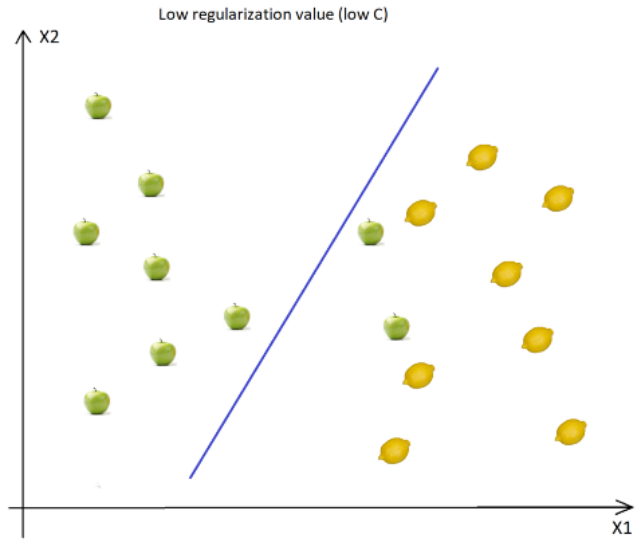
- So after the transformation, we can easily delimit the two classes using just a single line.
- In real life applications we won't have a simple straight line, but we will have lots of curves and high dimensions. In some cases we won't have two hyperplanes which separates the data with no points between them, so **we need some trade-offs, tolerance for outliers**. Fortunately the SVM algorithm has a so-called **regularization parameter** to configure the trade-off and to tolerate outliers.

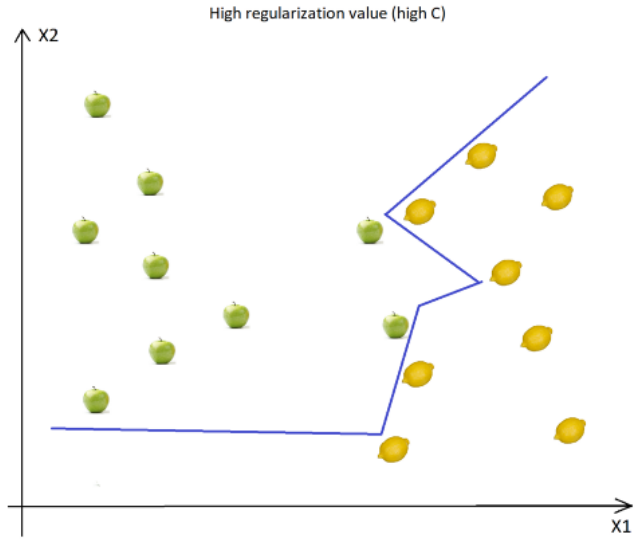
SVM

- **Tuning Parameters**
- As we saw in the previous section **choosing the right kernel is crucial**, because if the transformation is incorrect, then the model can have very poor results. As a rule of thumb, **always check if you have linear data** and in that case always **use linear SVM** (linear kernel). **Linear SVM is a parametric model**, but an **RBF kernel SVM isn't**, so the complexity of the latter grows with the size of the training set. Not only is **more expensive to train an RBF kernel SVM**, but you also have to **keep the kernel matrix around**, and the **projection into** this "infinite" **higher dimensional space** where the data becomes linearly separable is **more expensive** as well during prediction. Furthermore, you have **more hyperparameters to tune**, so model selection is more expensive as well! And finally, it's much **easier to overfit** a complex model!

SVM

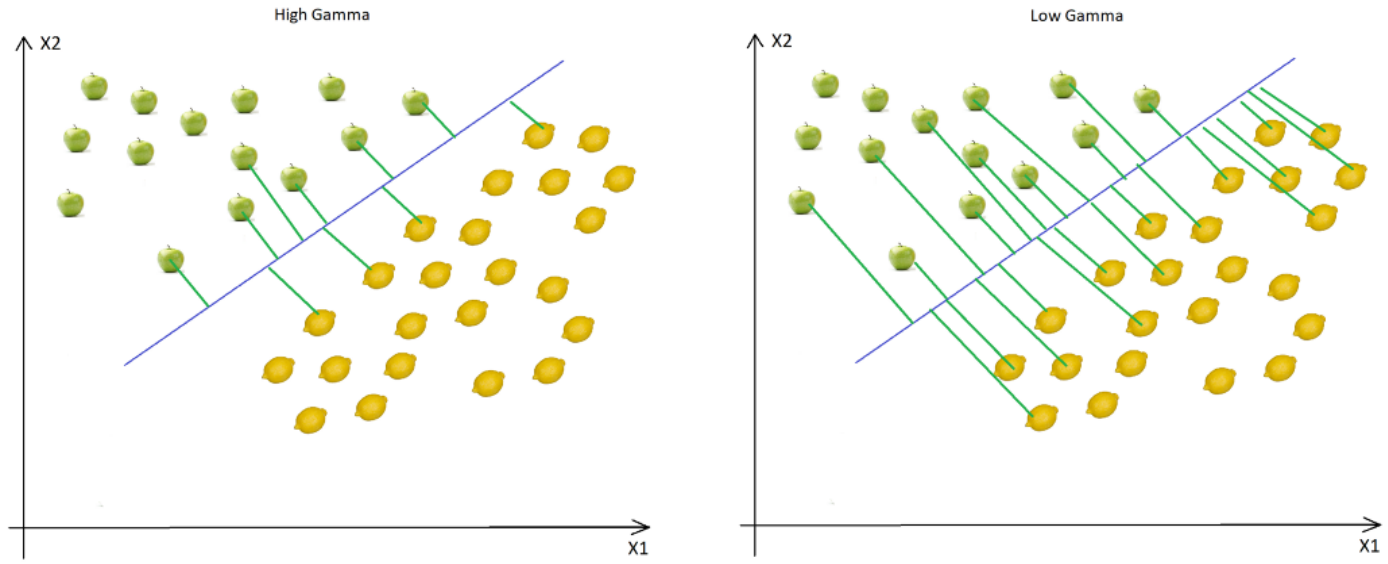
- **Regularization**
- The **Regularization Parameter** (**in python it's called C**) tells the SVM optimization **how much you want to avoid miss classifying** each training example.
- If the **C is higher**, the optimization will choose **smaller margin** hyperplane, so training data **miss classification rate will be lower**.
- On the other hand, if the **C is low**, then the **margin will be big**, even if there **will be miss classified** training data examples. This is shown in the following two diagrams:





SVM

- **Gamma**
- The next important parameter is **Gamma**. The gamma parameter defines **how far the influence of a single training example reaches**. This means that **high Gamma** will consider only points **close** to the plausible hyperplane and **low Gamma** will consider **points at greater distance**.



SVM

- As you can see, decreasing the Gamma will result that finding the correct hyperplane will consider points at greater distances so more and more points will be used (green lines indicates which points were considered when finding the optimal hyperplane).
- **Margin**
- The last parameter is the **margin**. We've already talked about margin, **higher margin results better model**, so better classification (or prediction). The margin should be always **maximized**.

SVM

- The most important parameters are:

1. **kernel:** the kernel type to be used. The most common kernels are **rbf** (this is the default value), **poly** or **sigmoid**, but you can also create your own kernel.
2. **C:** this is the **regularization parameter** described in the Tuning Parameters section
3. **gamma:** this was also described in the Tuning Parameters section
4. **degree:** it is used **only if the chosen kernel is poly** and sets the degree of the polinom
5. **probability:** this is a boolean parameter and if it's true, then the model will return for each prediction, the vector of probabilities of belonging to each class of the response variable. So basically it will give you the **confidences for each prediction**.
6. **shrinking:** this shows whether or not you want a **shrinking heuristic** used in your optimization of the SVM, which is used in Sequential Minimal Optimization. It's default value is true, an **if you don't have a good reason, please don't change this value to false**, because shrinking will **greatly improve your performance**, for very **little loss** in terms of **accuracy** in most cases.

SVM

Advantages

1. SVN can be very **efficient**, because it uses only a subset of the training data, only the support vectors
2. Works very well on **smaller data sets, on non-linear data sets** and **high dimensional spaces**
3. Is very **effective** in cases where **number of dimensions is greater than the number of samples**
4. It can have **high accuracy**, sometimes can perform even better than neural networks
5. **Not** very **sensitive to overfitting**

SVM

Disadvantages

1. **Training time is high** when we have large data sets
2. When the data set has more **noise** (i.e. target classes are overlapping) **SVM doesn't perform well**

Popular Use Cases

1. Text Classification
2. Detecting spam
3. Sentiment analysis
4. Aspect-based recognition
5. Aspect-based recognition
6. Handwritten digit recognition

NIET Greater Noida
Autonomous Institute
Estd.2001
Get Future Ready

K-Nearest Neighbor (KNN)

Definition — Instance-Based / Lazy Learning

KNN is a non-parametric, instance-based learner. Rather than building a model during training, it MEMORIZES the entire training set. To classify a new point, it finds the K closest training examples and assigns the MAJORITY class. Famous as a "lazy learner" — no training, all computation at prediction time.

Lazy vs Eager Learning

Lazy Learner (KNN)

- No training phase — just stores data.
- Decision is made AT QUERY TIME.
- Fast "training", slow prediction.
- Stores all training data (memory-intensive).
- Adapts immediately to new data.

Examples:
KNN, Case-Based Reasoning, Local regression.

Eager Learner

- Builds a generalized model during TRAINING.
- Decision is precomputed.
- Slow training, FAST prediction.
- Stores only learned model (compact).
- Must re-train for new data.

Examples:
Decision Trees, SVM, Neural Networks, Logistic Regression.

KNN Algorithm — Step by Step

The KNN Algorithm

1 — Store Training Data

Save all (x, y) training examples as-is. No model fitting.

2 — Choose K and Distance Metric

Pick K (number of neighbors). Choose distance function (Euclidean, Manhattan, etc.).

3 — For Each New Query Point

Compute distance from query to EVERY training point.

4 — Find K Nearest Neighbors

Select the K training points with smallest distances.

5 — Vote (Classification)

Assign the MAJORITY class among the K neighbors. Ties broken arbitrarily or by closest.

6 — Average (Regression)

For regression: predict the MEAN (or median) of neighbors' target values.

NIET Greater Noida
Autonomous Institute
Estd.2001
Get Future Ready

Distance Metrics

The choice of distance metric defines how "closeness" is measured — different metrics give different neighborhoods and predictions.

Euclidean (L2)

$$d(x, y) = \sqrt{\Sigma(x_i - y_i)^2}$$

Straight-line "as the crow flies" distance.
Default for KNN.

Use: *Continuous numerical features.*

Manhattan (L1)

$$d(x, y) = \Sigma\,|x_i - y_i|$$

Sum of absolute differences. "City-block" distance.
More robust to outliers.

Use: *Grid-like data, high dimensions.*

Minkowski (Lp)

$$d(x, y) = (\Sigma|x_i - y_i|^p)\text{^}(1/p)$$

GENERAL form. p=1 gives Manhattan;
p=2 gives Euclidean.

Use: *Flexible — tune p as hyperparameter.*

Hamming

$$d(x, y) = \#(x_i \neq y_i)$$

Count of positions where strings differ.
For categorical / binary features.

Use: *Text, DNA, categorical data.*

Always SCALE features before computing distances — otherwise large-scale features dominate.

Choosing the Value of K

K is the SINGLE most important hyperparameter in KNN. Too small → overfits to noise. Too large → underfits, smooths over class boundaries. Finding the right K is a bias-variance trade-off.

Small K (e.g. K=1, K=3)

- Highly LOCAL predictions.
- Very sensitive to noise / outliers.
- HIGH variance, low bias.
- Decision boundary is JAGGED.
- **Risk: OVERFITTING.**

Large K (e.g. K=N/2)

- GLOBAL, smoothed predictions.
- Robust to noise but misses fine patterns.
- Low variance, HIGH bias.
- Decision boundary is SMOOTH.
- **Risk: UNDERFITTING.**

How to Pick K — Best Practices

1. **Try K = √n** (square root of training size) as a heuristic starting point.
2. **Use ODD K** for binary classification to avoid tie votes.
3. **Cross-validation:** test K = 1, 3, 5, 7, ... up to ~20 and pick the value that maximizes validation accuracy.
4. **Plot the "Elbow":** error vs K — look for the "elbow" point where error stabilizes.

KNN — Pros & Cons

Advantages

- SIMPLEST ML algorithm to understand and implement.
- NO training phase — instant model.
- Handles MULTI-CLASS naturally.
- Non-parametric — no assumptions about data distribution.
- Can capture COMPLEX decision boundaries.
- Naturally adapts to new training data.

Disadvantages

- SLOW prediction on large datasets — O(N·d) per query.
- MEMORY-intensive — must store all training data.
- Curse of DIMENSIONALITY — distances lose meaning in high dim.
- Sensitive to feature SCALES — must standardize.
- Sensitive to IRRELEVANT features.
- Struggles with imbalanced classes.

Optimizations & Variants

KD-Tree

Space-partitioning data structure.
Speeds up search to O(log N) for low dim.

Ball Tree

Hierarchical clustering of points.
Better for higher dimensions.

Weighted KNN

Closer neighbors get HIGHER vote weight
(e.g. inverse distance).

Approximate NN

FAISS, Annoy — trade accuracy for speed.
Great for huge datasets.

NIET Greater Noida
Autonomous Institute
Estd.2001
Get Future Ready

KNN — Applications & When to Use

When to Use KNN

KNN shines when: (1) the dataset is SMALL or moderate, (2) decision boundaries are irregular / non-linear, (3) features have similar scales after standardization, (4) interpretability of "why" matters less than getting good local predictions. Avoid KNN on massive datasets or in very high dimensions.

Real-World Applications

Recommender Systems

"Users similar to you also liked..." — Netflix, Amazon item-based filtering.

Image Classification

Handwritten digit recognition (MNIST), face matching using pixel/feature distances.

Pattern Recognition

Bank-note authentication, OCR, fingerprint matching using similarity.

Anomaly Detection

Points with far neighbors are outliers — used in fraud detection.

Medical Diagnosis

Compare new patient to similar past cases — case-based reasoning.

Search & Retrieval

"Find me items similar to this" — content-based image search, document search.

Bottom line: *KNN is the "hello world" of ML — simple, interpretable, often surprisingly competitive on small clean datasets.*

Daily Quiz

1.Decision Tree
2.Entropy, Information Gain, Gini Impurity
3.Decision Tree Working For Categorical and Numerical Features
4.What are the scenarios where Decision Tree works well
5.Decision Tree Low Bias And High Variance- Overfitting
6.Hyperparameter Techniques
7.Library used for constructing decision tree
8.Impact of Outliers Of Decision Tree
9.Impact of mising values on Decision Tree
10.Does Decision Tree require Feature Scaling

MCQ

Question 1 :
SVM stands for?
Options :
a. Simple Vector Machine
b. Support Vector Machine
c. Super Vector Machine
d. All the Above

Question 2 :
SVM is classified into how many types?
Options :
a. One
b. Two
c. Three
d. Four

MCQ

Question 3 :
SVM, which best segregates classes into how many classes?
Options :
a. One
b. Two
c. Three
d. Four

Question 4 :
SVM is a supervised Machine Learning can be used for
Options :
a. Regression
b. Classification
c. Either a or b
d. None of These

Glossary Questions

1. A _____ is a decision support tool that uses a tree-like graph or model of decisions and their possible consequences, including chance event outcomes, resource costs, and utility.
a) Decision tree
b) Graphs
c) Trees
d) Neural Networks

2. Decision Tree is a display of an algorithm.
a) True
b) False

Glossary Questions

3. What is Decision Tree?
a) Flow-Chart
b) Structure in which internal node represents test on an attribute, each branch represents outcome of test and each leaf node represents class label
c) Flow-Chart & Structure in which internal node represents test on an attribute, each branch represents outcome of test and each leaf node represents class label
d) None of the mentioned

4. Decision Trees can be used for Classification Tasks.
a) True
b) False

NIET
Greater Noida
Autonomous Institute
Estd.2001
Get Future Ready

Assignment 1

1. Calculate the regression coefficient and obtain the lines of regression for the following data

X	1	2	3	4	5	6	7
Y	9	8	10	12	11	13	14

2. Calculate the two regression equations of X on Y and Y on X from the data given below, taking deviations from a actual means of X and Y.

Price(Rs.)	10	12	13	12	16	15
Amount demanded	40	38	43	45	37	43

Estimate the likely demand when the price is Rs.20.

NIET
Greater Noida
Autonomous Institute
Estd.2001
Get Future Ready

Assignment 2

- What is entropy?
- What is information gain?
- How are entropy and information gain related vis-a-vis decision trees?
- How do you calculate the entropy of children nodes after the split based on on a feature?
- How do you decide a feature suitability when working with decision tree?
- Explain feature selection using information gain/entropy technique?
- Which algorithm (packaged) is used for building models based on the decision tree?
- What are some of the techniques to decide decision tree pruning?

Assignment 1

1. What are Support Vector Machines (SVMs)?
2. What are Support Vectors in SVMs?
3. What is the basic principle of a Support Vector Machine?
4. What are hard margin and soft Margin SVMs?
5. What do you mean by Hinge loss?
6. What is the “Kernel trick”?

NIET
Greater Noida
Autonomous Institute
Estd.2001
Get Future Ready

Old Question Papers

Note: No old question paper available for this subject. Introduced first time.

I have added expected question for university exam in next slide.

Expected Questions for University Exam

- What is the role of the C hyper-parameter in SVM? Does it affect the bias/variance trade-off?
- Explain different types of kernel functions.
- How you formulate SVM for a regression problem statement?
- What affects the decision boundary in SVM?
- What is a slack variable?
- What is a dual and primal problem and how is it relevant to SVMs?

Recap of Unit

- **A decision tree is a tree-shaped diagram that shows statistical probability or determines a course of action**. It shows the analysts and by that, the decision-makers which steps they have to take and how different choices could affect the whole process.
- **SVM is a supervised learning algorithm which separates the data into different classes through the use of a hyperplane**. The chosen hyperplane is one with the greatest margin between the hyperplane and all points, this yields the greatest likelihood of accurate classification.